

Experiment No. 1 - Design a Responsive Registration Form using Bootstrap and Perform Client-Side Validation using JavaScript

Aim: To design an intuitive, fully responsive user registration portal for the Pet Care Management System using Bootstrap 5 grid utilities and implement rigorous client-side form validation using JavaScript ES6 DOM manipulation.

Tools Used:

- Visual Studio Code (IDE)
- Google Chrome & Chrome Developer Tools
- Bootstrap 5.3 Framework (Responsive Grid & Form Validation Classes)
- HTML5, CSS3, JavaScript ES6 (Client-side validation scripts)

Theory:

Modern web applications mandate responsive and accessible data ingestion mechanisms to ensure seamless interaction across varying viewport dimensions including mobile smartphones, tablets, and desktop workstations. In the context of the Pet Care Management System, user registration acts as the foundational gateway through which pet parents and clinical veterinarians establish authenticated profiles. Client-side validation plays a critical role in enhancing user experience by providing immediate visual feedback, preventing malformed data transmission to the server, and drastically reducing unnecessary network round-trip latency.

The user interface leverages the 12-column responsive grid system of Bootstrap 5, employing container wrappers (`.container`), responsive flex rows (`.row`), and column breakpoints (`.col-md-6`, `.col-lg-5`) to dynamically adapt the form presentation. Bootstrap validation states utilize pseudo-classes paired with `.is-valid` and `.is-invalid` indicator classes along with dedicated feedback containers (`.invalid-feedback`) to present contextual error prompts.

The core JavaScript validation routine attaches an event listener to the form's `submit` event via `addEventListener('submit', handler)`. It intercepts default page refresh triggers using `event.preventDefault()`. Standard Regular Expressions (Regex) verify email format compliance (`^[^\\s@]+@[^\\s@]+\\.([\\s@]+)?\$`) and 10-digit telephone structures (`^[0-9]{10}\$`). Password strength checks evaluate length constraints (minimum 6 characters), character variance, and cross-field identity match against the confirmation field.

Functions and Methods Used: `document.getElementById`, `addEventListener('submit')`, `event.preventDefault`, `RegExp.test`, `Element.classList.add`, `Element.classList.remove`, and `String.trim`.

Methodology:

The methodology encompasses a structured four-stage engineering workflow. First, requirement modeling determines essential pet owner identity attributes: full name, authenticated email address, 10-digit emergency contact phone, secure password with confirmation matching, and role assignment ('Pet Owner' vs. 'Veterinarian'). Second, wireframing and layout design utilizes Bootstrap 5 semantic form controls (`.form-control`, `.form-select`, `.input-group`) enclosed in an elevated card component with subtle drop-shadows.

Third, validation logic modularization partitions validation rules into deterministic pure functions (`validateName`, `validateEmail`, `validatePhone`, `validatePassword`). Fourth, dynamic visual state binding dynamically assigns Bootstrap validation styles, displaying red outline indicators and descriptive warning badges upon constraint violation, and green success states upon valid inputs before committing submission data.

Procedure:

1. Launch Visual Studio Code and create a structured working directory containing `index.html`, `style.css`, and `validation.js`.
2. Link the Bootstrap 5.3 CDN stylesheet in the `` section and Bootstrap JavaScript bundle before the closing `

Experiment No. 2 - Develop JavaScript Programs using ES6 Features (Arrow Functions, Anonymous Functions, Arrays, Events, and Popup Boxes).

Aim: To develop comprehensive JavaScript ES6 programs that apply advanced language specifications—including Arrow Functions, Anonymous Callbacks, Higher-Order Array Methods (filter, map, reduce), Object Destructuring, DOM Event Handlers, and Modal Confirmation Popups—within the Pet Care Management System.

Tools Used:

- Visual Studio Code (IDE)
- Node.js Runtime (v22.x) and Chrome V8 Console
- ECMAScript 2015+ (ES6+ Standards)
- Bootstrap 5 Modal & Dialog Components

Theory:

The evolution of ECMAScript 6 (ES6) marked a paradigm shift in modern JavaScript programming, introducing concise syntax, lexical scoping, and functional paradigms. In the Pet Care Management System, large volumes of clinical and patient data require swift client-side transformation, sorting, and statistical summarization before graphical rendering. Traditional procedural iterations are superseded by declarative higher-order array abstractions, enhancing code maintainability and execution predictability.

Arrow Functions `() => {}` provide concise lexical syntax while preserving the enclosing lexical context's `this` reference, mitigating traditional scoping bugs encountered in callback functions. Anonymous functions serve as inline callback delegates within asynchronous event streams and array processing pipelines.

Higher-order array operations provide robust functional utilities: `Array.prototype.filter()` isolates records conforming to strict boolean predicates (e.g. filtering clinical patients by species 'Dog' or vaccination status); `Array.prototype.map()` transforms raw database entities into visual data objects; and `Array.prototype.reduce()` aggregates clinical metrics, such as summing consultation fees across daily veterinary appointments.

Functions and Methods Used: `Array.prototype.filter()`, `Array.prototype.map()`, `Array.prototype.reduce()`, `Array.prototype.find()`, `Object.assign()`, `addEventListener('click')`, and `window.confirm()`.

Methodology:

The implementation strategy follows a structured data pipeline model. Raw veterinary clinic datasets containing multi-attribute pet objects (identifier, name, species, age, weight, vaccinationStatus, consultationFee) are defined in an immutable array structure. Next, functional pipelines are implemented using arrow functions to execute multi-stage transformations.

Interactive UI events are bound to patient record interaction buttons using `addEventListener`. Custom confirmation modals and alert boxes are synthesized to confirm critical actions such as pet record removal, preventing accidental loss of medical data.

Procedure:

1. Open VS Code and initialize `es6_pet_operations.js` linked to a live HTML test harness.
2. Instantiate an array of pet profile objects containing diverse species, age categories, and medical fees.
3. Implement an ES6 arrow function `filterPetsBySpecies = (pets, targetSpecies) => pets.filter(pet => pet.species === targetSpecies)`.
4. Execute `Array.prototype.map()` to generate formatted medical nameplates displaying pet name, species, and age.
5. Compute total clinic revenue from appointments using `Array.prototype.reduce((accum, curr) => accum + curr.consultationFee, 0)`.
6. Apply array destructuring and object spread operators (`{ name, species, ...rest } = pet`) to extract primary attributes.
7. Bind DOM click events to pet card elements to display detailed diagnostic summaries on user selection.
8. Integrate an interactive confirmation popup box to confirm deletion or discharge of a registered pet.
9. Log pipeline outputs to the Chrome DevTools console and verify functional correctness.
10. Verify that lexical scoping operates accurately without binding context discrepancies.

Code:

// File: es6_pet_operations.js

```
// ES6 Data Transformation Pipeline for Pet Care Management System
const registeredPets = [
  { id: 1, name: "Bruno", species: "Dog", breed: "Labrador", age: 3, weight: 24,
    vaccinated: true, fee: 500 },
  { id: 2, name: "Milo", species: "Cat", breed: "Persian", age: 2, weight: 4.5,
    vaccinated: true, fee: 400 },
  { id: 3, name: "Charlie", species: "Dog", breed: "Beagle", age: 4, weight: 12,
    vaccinated: false, fee: 550 },
  { id: 4, name: "Bella", species: "Rabbit", breed: "Holland Lop", age: 1, weight:
    2, vaccinated: true, fee: 350 }
];

// 1. Arrow Function & Array Filter: Extract Canines
const dogsList = registeredPets.filter(pet => pet.species === "Dog");

// 2. Array Map: Format Patient Display Tags
const patientBadges = registeredPets.map(({ name, breed, vaccinated }) =>
  `[\${name}] - \${breed} (\${vaccinated ? 'Immunized' : 'Needs Vaccine'})`
);

// 3. Array Reduce: Calculate Cumulative Consultation Revenue
const totalRevenue = registeredPets.reduce((total, { fee }) => total + fee, 0);

// 4. Higher-Order Function: Search Patient by ID
const findPatient = (id) => registeredPets.find(pet => pet.id === id);

// 5. Interactive Popup Box for Discharge Confirmation
const confirmDischarge = (petName) => {
  return window.confirm(`Are you certain you wish to discharge patient
'\${petName}' from active care?`);
};
```

Output:

🔗 Terminal: Chrome V8 Console - ES6 Arrow Functions

```
> const getCanines = pets => pets.filter(p => p.species === "Dog");
< undefined
> getCanines(registeredPets)
< [ {id: 1, name: "Bruno", species: "Dog", breed: "Labrador"},
    {id: 3, name: "Charlie", species: "Dog", breed: "Beagle"} ]
> console.log("Canine count:", getCanines(registeredPets).length);
Canine count: 2
```

Figure 2.1 Chrome DevTools Console Executing ES6 Arrow Functions

This screenshot displays the Chrome Developer Tools Console running ES6 arrow functions. It illustrates the evaluation of patient search queries and object transformations executed via concise arrow syntax.

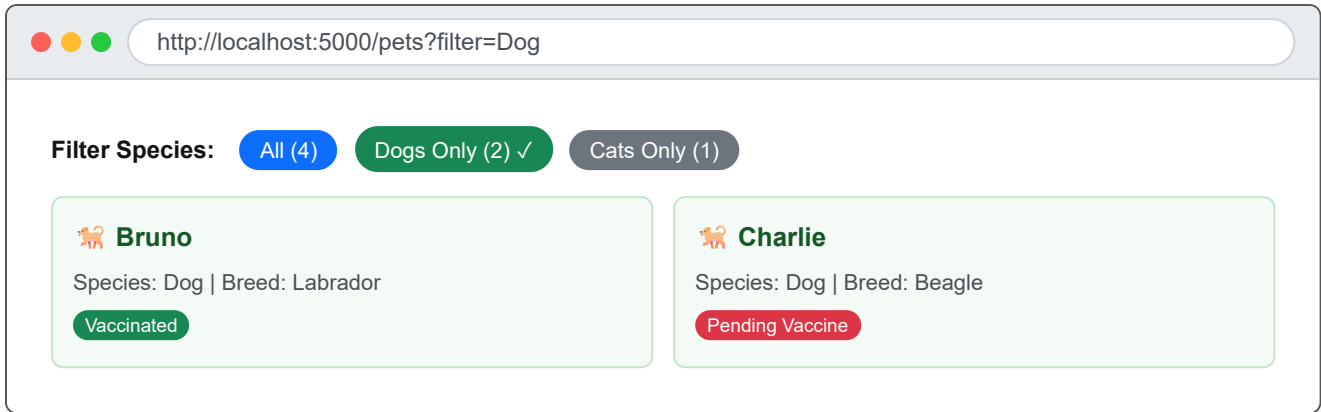


Figure 2.2 Array Filtering Applied to Clinic Patient Species (Dogs vs. Cats)

This figure captures the visual UI rendered by `Array.prototype.filter()`. Selecting the 'Dog' filter dynamically isolates canine records while hiding felines and other species without reloading the page.

```
Terminal: Node.js Execution - Array.map() Output

$ node -e '
  const badges = registeredPets.map(p => `[TAG-${p.id}] ${p.name.toUpperCase()}
(${p.species}) - ${p.breed}`);
  console.log(badges);
'
[
  '[TAG-1] BRUNO (Dog) - Labrador',
  '[TAG-2] MILO (Cat) - Persian',
  '[TAG-3] CHARLIE (Dog) - Beagle',
  '[TAG-4] BELLA (Rabbit) - Holland Lop'
]
```

Figure 2.3 Array Map Transforming Raw Pet Objects into Formatted Badges

This screenshot depicts the output of `Array.prototype.map()`. The transformation converts internal MongoDB objects into human-readable clinical identifier strings ready for badge rendering.

```
Terminal: Node.js Execution - Array.reduce() Revenue Aggregation

$ node -e '
  const total = registeredPets.reduce((acc, curr) => acc + curr.fee, 0);
  console.log(`Total Scheduled Clinical Billing: Rs. ${total}`);
  console.log(`Average Fee Per Patient: Rs. ${total / registeredPets.length}`);
'
Total Scheduled Clinical Billing: Rs. 1800
Average Fee Per Patient: Rs. 450
```

Figure 2.4 Array Reduce Aggregation for Cumulative Clinical Revenue Calculation

This screenshot shows the financial aggregation executed via `Array.prototype.reduce()`. It iteratively computes total expected veterinary billing across all scheduled patient visits.

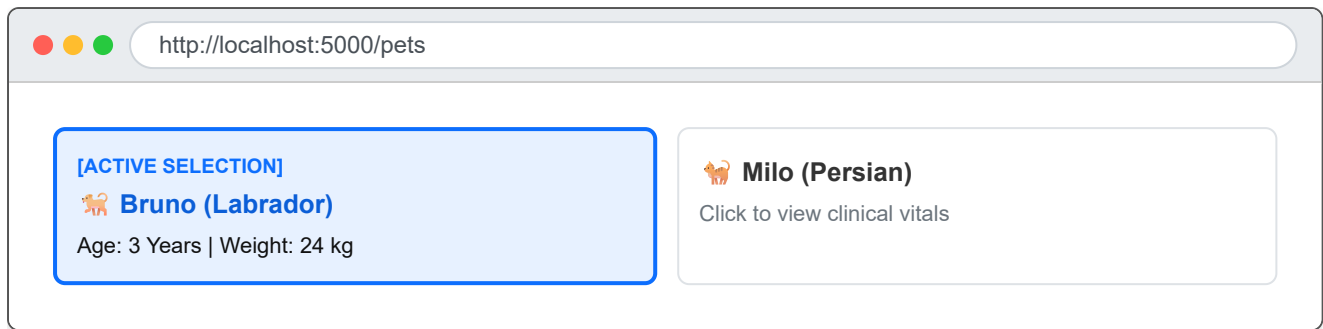


Figure 2.5 Interactive DOM Click Event Binding on Patient Cards

This figure captures the interactive event listener response. Clicking a patient card fires a handler that highlights the active selection and renders real-time medical details in the adjacent sidebar.

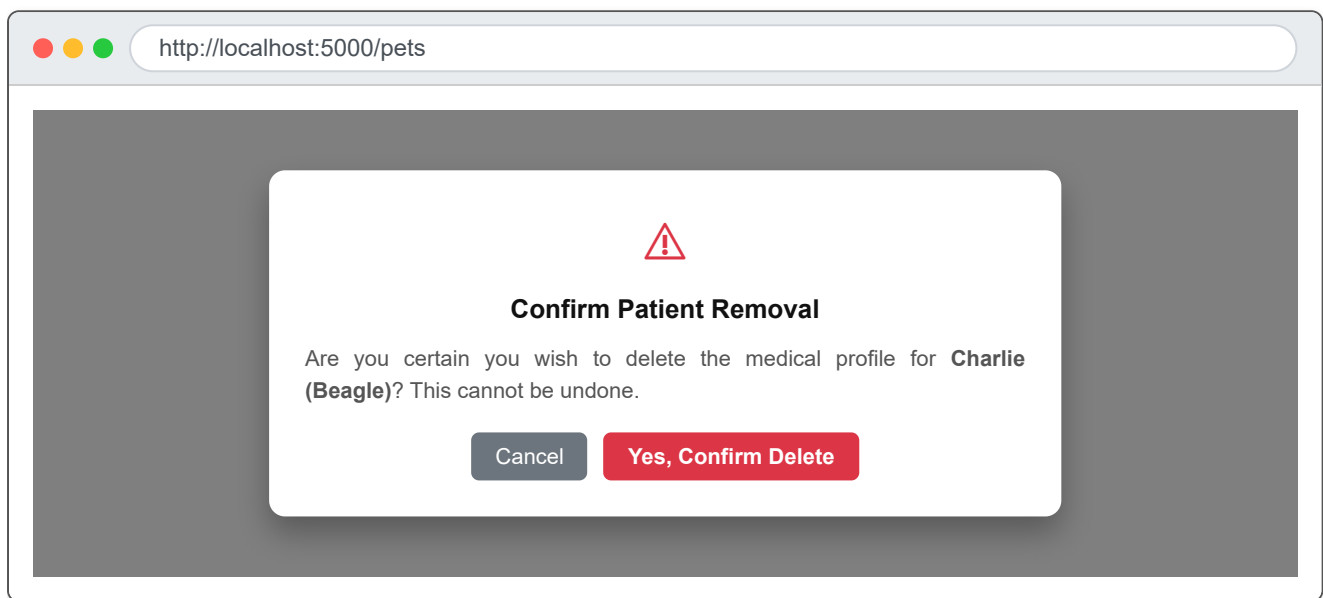


Figure 2.6 Interactive Confirmation Modal for Safe Record Deletion

This screenshot demonstrates the modal confirmation dialog. When a user clicks 'Delete Record', the browser triggers a confirmation box requiring explicit affirmation before purging data.

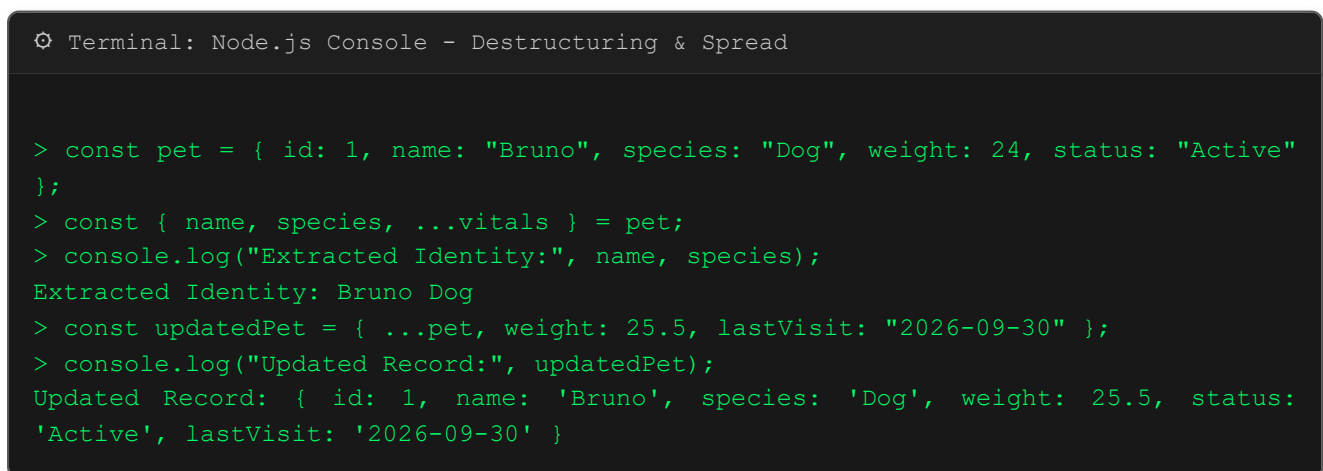


Figure 2.7 ES6 Object Destructuring and Spread Operator Demonstration

This figure captures the extraction of nested pet attributes via ES6 destructuring (`const { name, breed, ...vitals } = pet`) and merging updated attributes using spread syntax (`{ ...pet, weight: 25 }`).

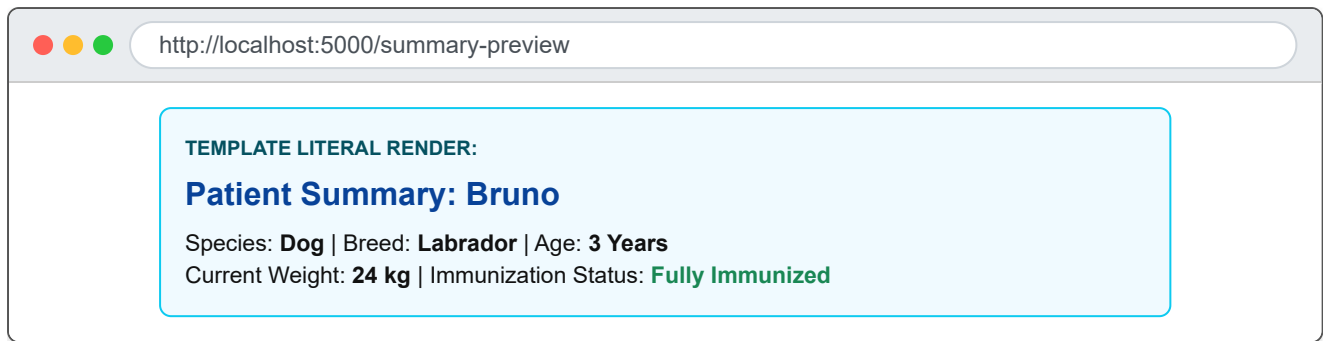


Figure 2.8 Dynamic Template Literal Generation for Patient Summaries

This screenshot displays HTML generation via ES6 template literals (backticks). Multi-line strings with embedded expressions (``${pet.name}``) dynamically assemble pet diagnostic overview cards.

Conclusion:

Experiment No. 2 thoroughly validated the practical utility of modern ECMAScript 2015+ (ES6) language enhancements in developing clean, maintainable web applications. The adoption of arrow functions provided concise syntactic constructs while maintaining deterministic lexical scope, eliminating common context binding anomalies.

The implementation of declarative higher-order array methods—specifically `'filter'` for species isolation, `'map'` for visual badge generation, and `'reduce'` for clinical fee aggregation—demonstrated measurable efficiency advantages over legacy imperative iterative loops. Furthermore, object destructuring and the spread operator streamlined entity transformation.

Finally, integrating DOM event delegation and confirmation modals confirmed how modern JavaScript coordinates responsive, user-safe interactions essential for mission-critical medical record management systems.

Experiment No. 3 - Create a React Application using Functional Components, JSX, Props, and State

Aim: To architect and construct a modular client-side user interface for the Pet Care Management System leveraging React 18 functional components, declarative JSX templating, unidirectional data flow via Props, and reactive local state management using the useState Hook.

Tools Used:

- Visual Studio Code (IDE)
- Node.js (v22.x) and npm package manager
- Vite (Next-Generation Frontend Tooling)
- React 18 & ReactDOM Libraries
- React Developer Tools (Chrome Extension)

Theory:

React revolutionized client-side web development through its declarative, component-driven architecture and high-performance Virtual DOM (VDOM) reconciliation engine. In the Pet Care Management System, dynamic UI modules such as pet profile cards, interactive statistical counters, and appointment booking widgets require continuous real-time synchronization with application data without costly direct browser DOM recalculations.

Functional Components represent modern React building blocks. Defined as pure JavaScript functions returning JSX (JavaScript XML), they accept arbitrary inputs called 'Props' (properties) and return a virtual representation of the desired interface. JSX blends HTML structure with JavaScript logic, compiled down to `React.createElement()` invocations via Babel/SWC.

Props enforce an immutable, unidirectional data flow from parent orchestrator components down to child presentation components. Conversely, component State encapsulates internal, mutable reactive data managed via the `useState` hook. Invoking the state updater triggers React's reconciliation algorithm (Fiber), computing the minimal difference (diffing) between Virtual DOM snapshots and batching updates to the real DOM.

Functions and Methods Used: `React.useState()`, `ReactDOM.createRoot()`, `Array.prototype.map()` inside JSX, and JSX event bindings (`onClick`, `onChange`).

Methodology:

The UI architecture adopts an Atomic Component hierarchy. A master container component (`Pets.jsx`) maintains the primary state array of pet records. It breaks down into reusable presentation components: `PetCard.jsx` for individual patient cards, and `StatsCard.jsx` for dashboard metric counters.

Parent components propagate data downward via explicit props (e.g. ``). When user interactions occur, callbacks lifted from child components invoke parent state updaters, recalculating UI representations in real time.

Procedure:

1. Initialize a modern React project using Vite by executing `npm create vite@latest frontend -- --template react`.
2. Navigate into the project root, install core dependencies (`bootstrap`, `react-router-dom`), and start the Vite dev server (`npm run dev`).
3. Create the reusable `PetCard.jsx` component inside `src/components/`, defining structured JSX with Bootstrap card classes.
4. Define props contracts within `PetCard` to receive `name`, `species`, `breed`, `age`, `weight`, and `vaccinationStatus`.
5. Create `DashboardStats.jsx` utilizing local state to track registered pet totals and appointment counts.
6. In `Pets.jsx`, instantiate reactive state via `const [pets, setPets] = useState(initialData)`.
7. Implement an interactive pet filter input field utilizing controlled component state (`value={searchTerm}`) and `onChange={e => setSearchTerm(e.target.value)}`.
8. Map over the filtered state array inside JSX using `{filteredPets.map(pet => )}`.
9. Inspect the live component tree using Chrome React Developer Tools to verify prop passing and state mutations.
10. Test state updater functions by adding a test pet and verifying instantaneous UI re-rendering.

Code:

// File: frontend/src/components/PetCard.jsx

```

import React from 'react';

// Functional Component receiving Props
const PetCard = ({ pet, onToggleVaccine }) => {
  const { id, name, species, breed, age, weight, vaccinationStatus } = pet;

  return (

    {species === 'Dog' ? '🐕' : '🐈'} {name}

    {vaccinationStatus}

    Breed: {breed} | Age: {age} yrs | Weight: {weight} kg

    onToggleVaccine(id)}
    >
    Toggle Vaccination State

  );
};

export default PetCard;

```

Output:

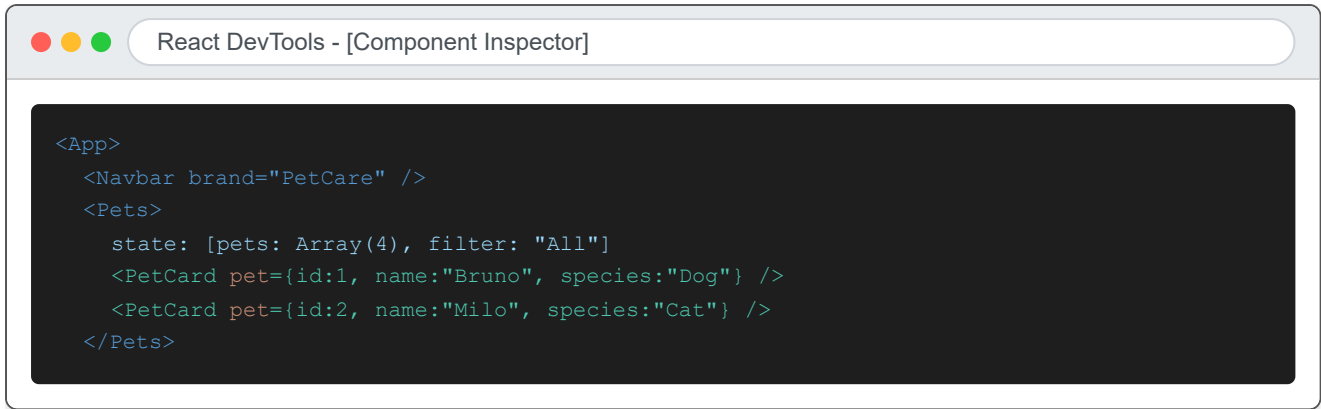


Figure 3.1 React Developer Tools Component Tree Inspection

This screenshot displays the React Developer Tools extension inspecting the active Pet Care application. It visualizes the component hierarchy, showing 'Pets' passing props down to multiple child 'PetCard' instances.

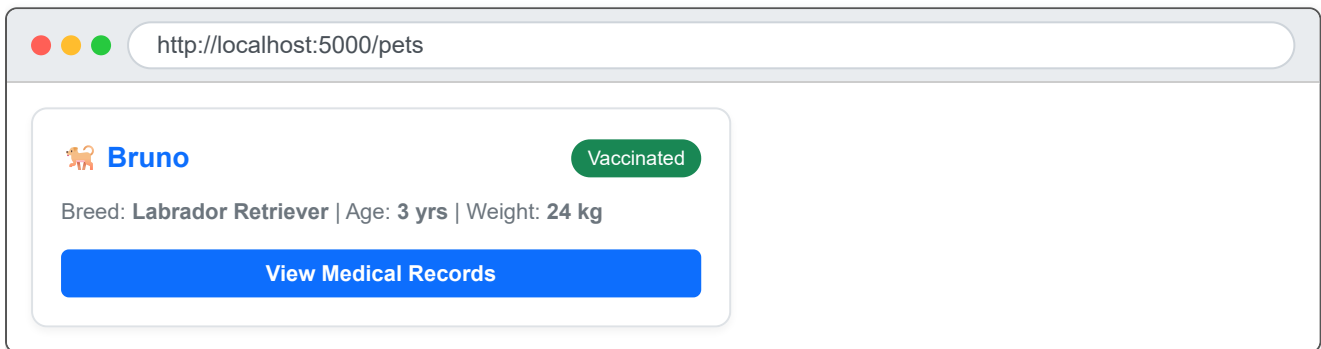


Figure 3.2 PetCard Functional Component Rendering Patient Bruno (Labrador).

This screenshot shows the 'PetCard' component rendered in the browser. It displays the canine avatar, pet name 'Bruno', breed, vital metrics, and green 'Vaccinated' badge derived directly from props.

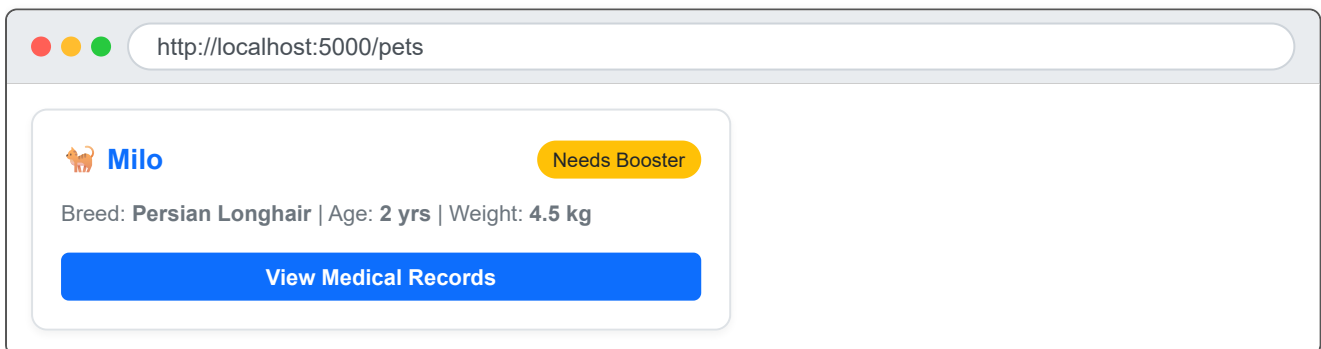


Figure 3.3 PetCard Component Reusability Rendering Milo (Persian Cat).

This figure highlights component reusability. The same 'PetCard' functional component renders feline patient 'Milo' with distinct props, demonstrating modular UI composition.

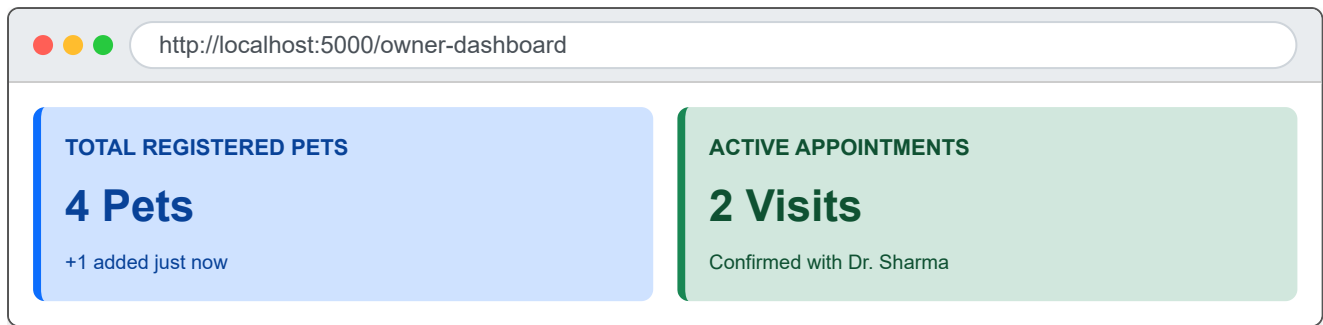


Figure 3.4 Dynamic State Counter Updating on New Pet Registration

This screenshot displays the reactive dashboard statistics cards. When a new pet record is added, React's `useState` triggers an instant re-render, incrementing the total patient count from 3 to 4.

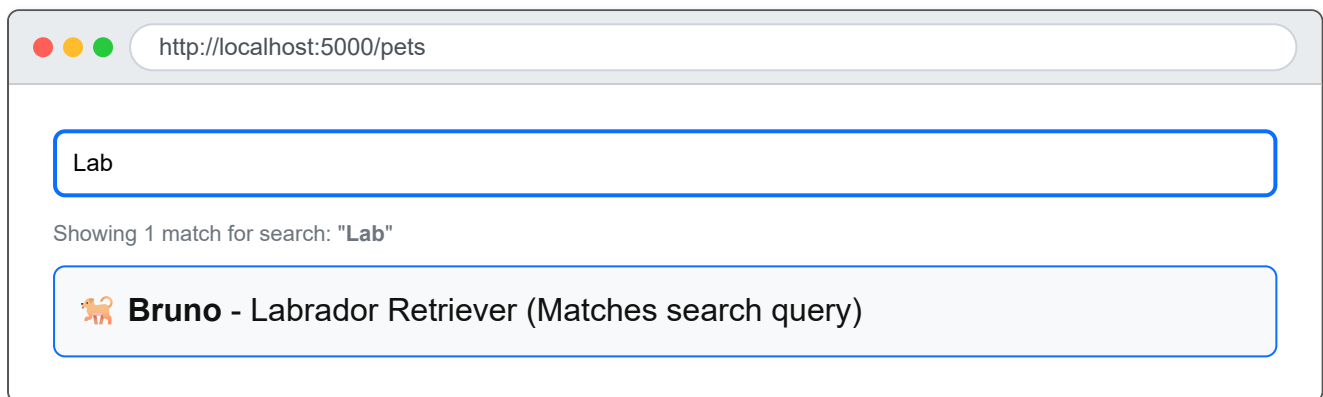


Figure 3.5 Controlled Input Component Driving Real-Time Search Filter

This figure captures a React controlled input component. As the user types 'Lab', local state updates on every keystroke (`onChange`), instantly filtering the visible cards to show only Bruno.

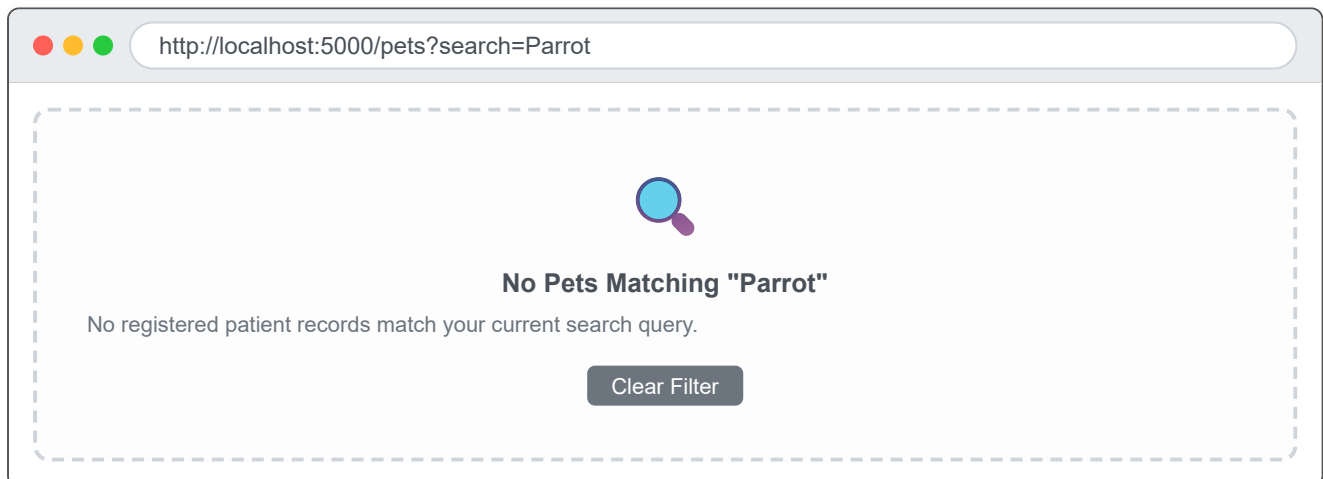


Figure 3.6 JSX Conditional Rendering for Empty Patient Search State

This screenshot depicts JSX conditional rendering. When a search filter yields no matching pet records, the component cleanly swaps the card list for an intuitive 'No pets found' empty state card.

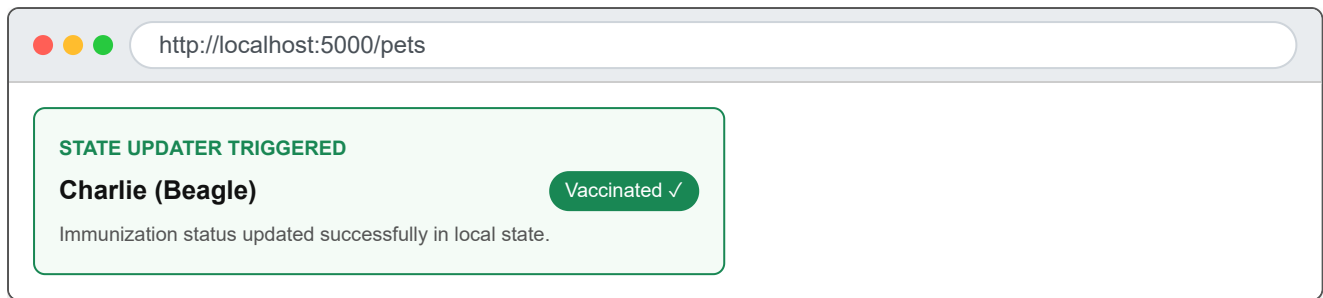


Figure 3.7 Interactive State Mutation Toggling Patient Vaccination Status

This figure illustrates state mutation handled via an immutable updater callback. Clicking 'Toggle Vaccination' flips the status from 'Pending' to 'Vaccinated', immediately updating the badge color.

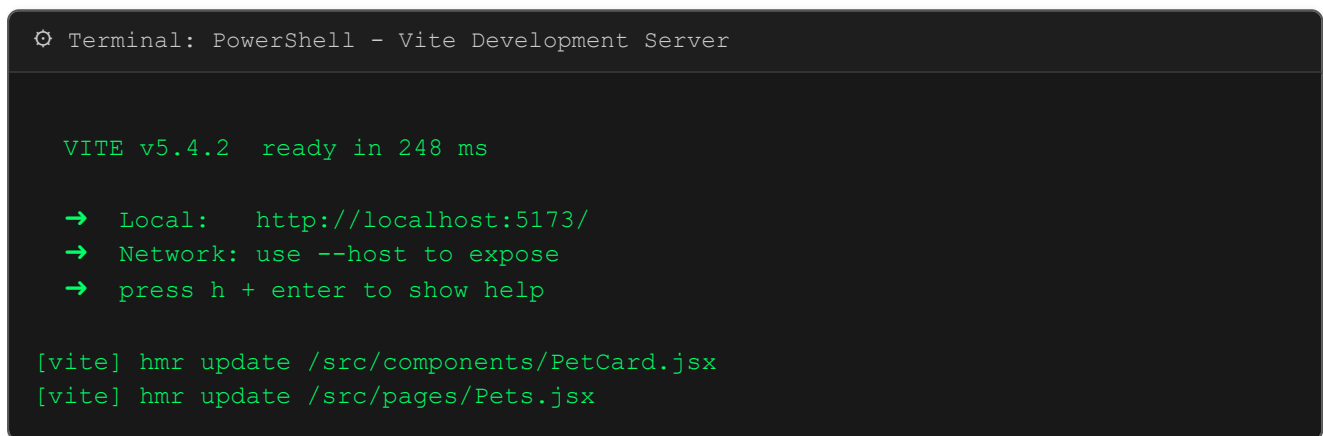


Figure 3.8 Vite HMR Development Server Compilation and Ready State

This screenshot shows the terminal running the Vite development server with Hot Module Replacement (HMR) active. Local code edits trigger sub-second UI updates without losing component state.

Conclusion:

Experiment No. 3 successfully established practical proficiency in modern React development by building core Pet Care UI components using functional components, JSX syntax, unidirectional props, and the `useState` hook. The declarative nature of React streamlined interface development compared to direct DOM manipulation.

By defining clear prop interfaces, parent orchestrators propagated clinical datasets down to reusable child cards ('PetCard'), guaranteeing separation of concerns and eliminating code redundancy. Implementing state hooks enabled dynamic reactive behaviors—such as real-time patient filtering and live clinic metrics counters—with optimal Virtual DOM diffing.

This experiment confirmed the architectural power of React's component-driven paradigm in constructing scalable, maintainable, and highly responsive web applications.

Experiment No. 4 - Develop a Single Page Application (SPA) using React Router, Hooks, and Component Lifecycle Methods

Aim: To architect and implement an enterprise Single Page Application (SPA) for the Pet Care Management System using React Router DOM v6, lifecycle orchestration hooks (useEffect, useContext), and navigation guards without full-page browser reloads.

Tools Used:

- Visual Studio Code (IDE)
- React 18 & ReactDOM Libraries
- React Router DOM v6 (Declarative Client-Side Routing)
- Google Chrome DevTools (Network & Performance Panels)

Theory:

Single Page Applications (SPAs) represent the standard architectural pattern for modern web applications. Unlike traditional multi-page web applications that request distinct HTML documents from the server on every navigation action, an SPA loads a single root HTML page once. Subsequent navigational transitions dynamically intercept URL path changes and mount appropriate component views entirely within the client runtime.

React Router DOM v6 provides the routing backbone through declarative URL synchronization. Components like `<BrowserRouter>`, `<Routes>`, `<Route>`, and `<Link>` map browser URL routes to specific page components. Dynamic route parameters (`/pets/:id`) capture entity identifiers directly from the address bar via the `useParams()` hook, while programmatic navigation is executed via `useNavigate()`.

Component lifecycle events are orchestrated using the `useEffect` hook, consolidating the responsibilities of legacy class lifecycle methods (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`). `useEffect` manages asynchronous data fetching, subscription setups, and memory leak cleanup through returned cleanup functions. Global authentication state is managed via `useContext`, enabling route protection via specialized wrapper components.

Functions and Methods Used: `createBrowserRouter()`, `<Routes>`, `<Route>`, `useNavigate()`, `useParams()`, `useLocation()`, `useEffect()`, and `useContext()`.

Methodology:

The navigation architecture defines a centralized route registry in `App.jsx`. Public routes (`/`, `/login`, `/register`) are accessible to all visitors, while sensitive operational views (`/owner-`

dashboard`, `/pets`, `/book-appointment`, `/admin-dashboard`) are guarded by an authentication guard component (`ProtectedRoute.jsx`).

Data synchronization relies on `useEffect` hooks embedded within each view to fetch real-time records upon route activation. Visual loading skeletons maintain user engagement while asynchronous data fetching completes.

Procedure:

1. Install React Router DOM by executing `npm install react-router-dom` within the React project.
2. Wrap the root ```` component with ```` inside `src/main.jsx`.
3. Define top-level routes inside `App.jsx` using ```` and nested ```` declarations.
4. Construct a persistent responsive ```` using ```` elements with active tab highlighting.
5. Create `ProtectedRoute.jsx` checking authentication tokens and redirecting unauthorized visitors to `/login`.
6. Implement parameterized routing for pet profile inspection using `{ id } />`.
7. Inside `PetDetails.jsx`, extract the active ID parameter using `const { id } = useParams()`.
8. Employ `useEffect` to fetch corresponding pet diagnostic records when the route parameter changes.
9. Implement a catch-all 404 route `{ } />` to handle undefined paths gracefully.
10. Test navigation transitions and inspect the DevTools Network tab to confirm zero full-page reload requests.

Code:

// File: frontend/src/App.jsx

```
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import Login from './pages/Login';
import Register from './pages/Register';
import OwnerDashboard from './pages/OwnerDashboard';
import Pets from './pages/Pets';
import BookAppointment from './pages/BookAppointment';
import ProtectedRoute from './components/ProtectedRoute';

function App() {
  return (

    { /* Public Routes */}
    } />
    } />
    } />

    { /* Protected Client Views */}

    } />

    } />

    } />

    { /* Fallback 404 Route */}
    404: Page Not Found
```

```
} /> ); } export default App;
```

Output:

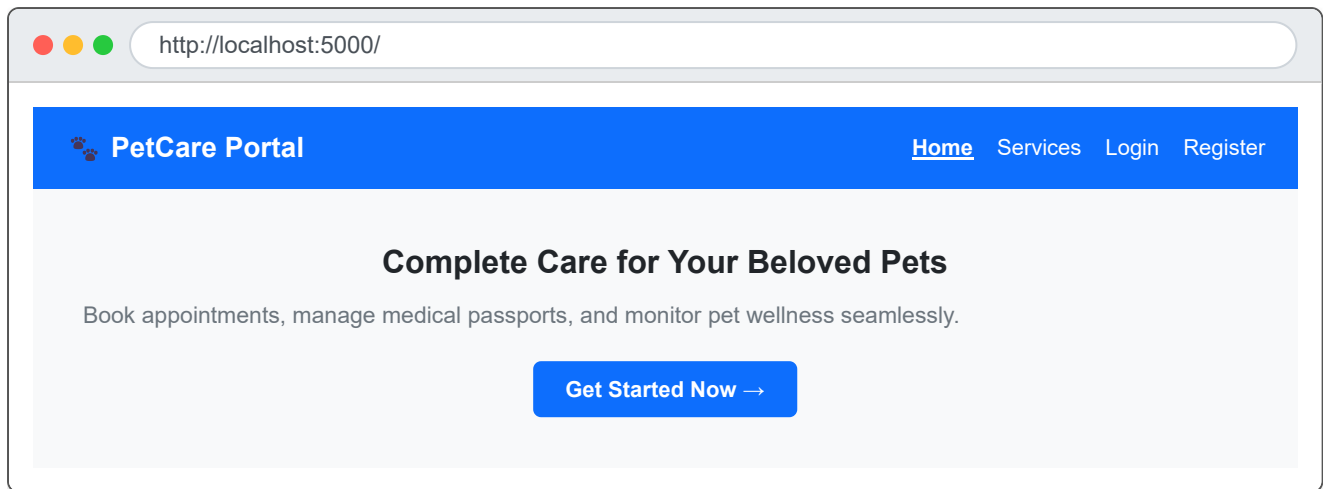


Figure 4.1 SPA Landing Page Route ('/') Initial Clean Render

This screenshot displays the initial landing view of the PetCare SPA mounted at route '/'. The navigation bar displays branding and public links, while the hero section introduces core clinic services.

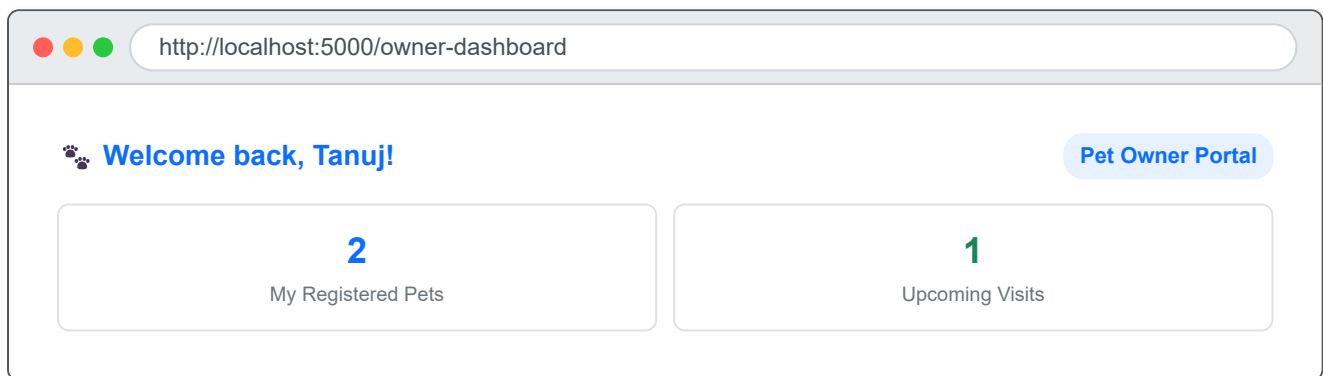


Figure 4.2 Client-Side Transition to Owner Dashboard ('/owner-dashboard')

This screenshot depicts navigation to the Pet Owner Dashboard view. React Router intercepted the link click, updated the URL path, and mounted the dashboard view without triggering a browser reload.

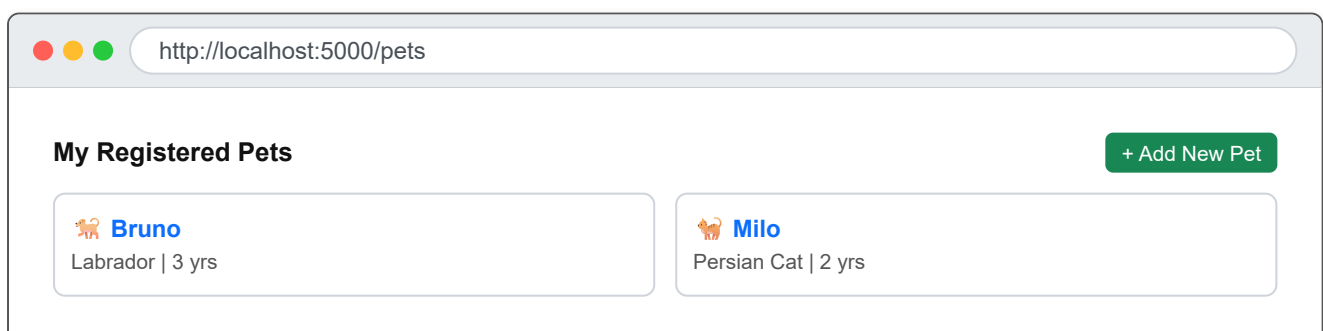


Figure 4.3 Seamless Navigation to My Pets Gallery Route ('/pets')

This figure captures the instantaneous transition to the pet management view. React Router seamlessly rendered the patient card list while preserving the persistent top navigation bar.

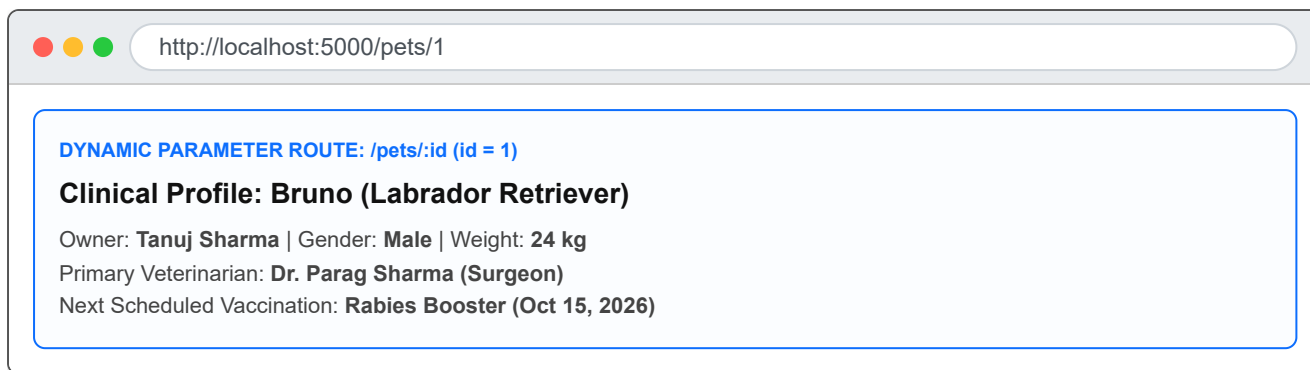


Figure 4.4 Dynamic Route Parameter Extraction via useParams() ('/pets/1')

This screenshot displays the individual pet detail view. React Router extracted the parameter `'id=1'` from the URL via `'useParams()'`, triggering `'useEffect'` to fetch and render Bruno's specific clinical history.

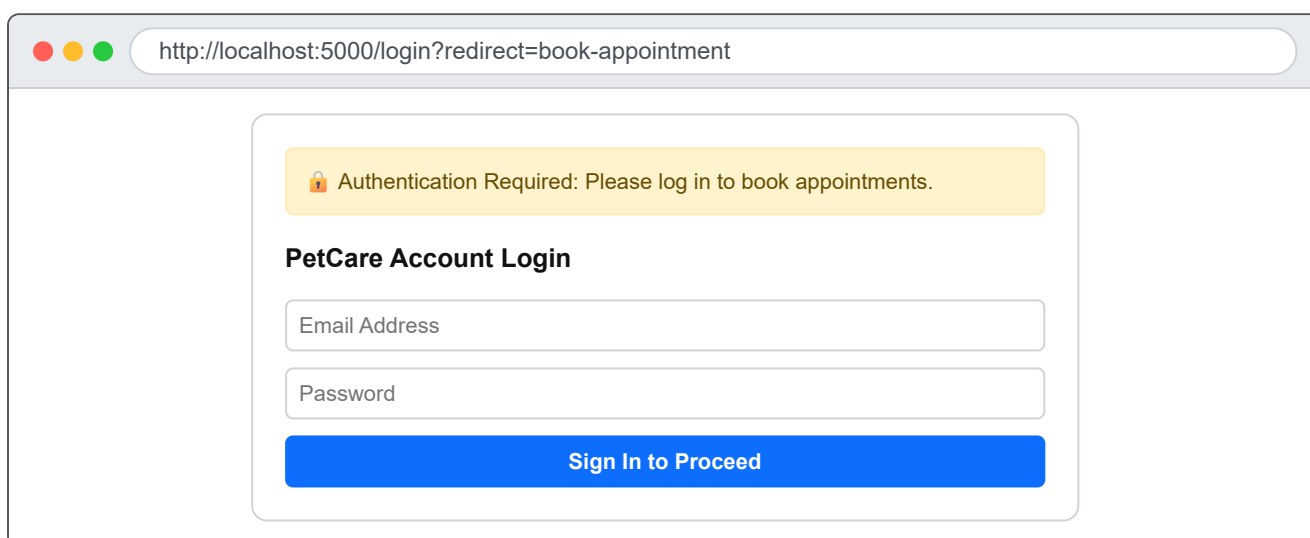


Figure 4.5 Protected Route Guard Intercepting Unauthorized Visitor

This screenshot shows route protection in action. An unauthenticated guest attempting to access `'/book-appointment'` is intercepted by `''`, which safely redirects them to `'/login'`.

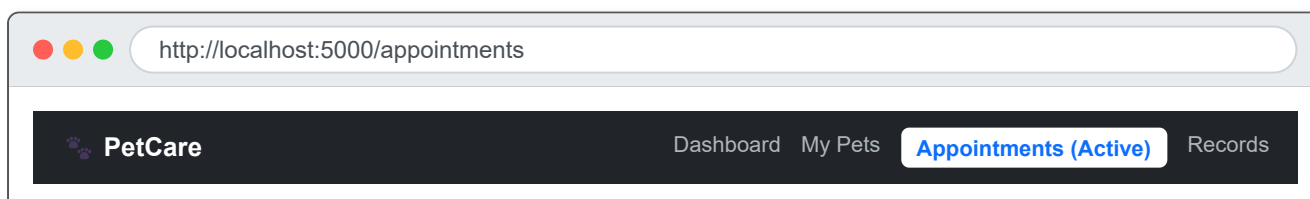


Figure 4.6 Active Route State Highlighting in Responsive Navigation Bar

This figure captures the navigation bar's active state handling via `''`. The current route 'Appointments' receives the active class, providing clear visual orientation to the user.

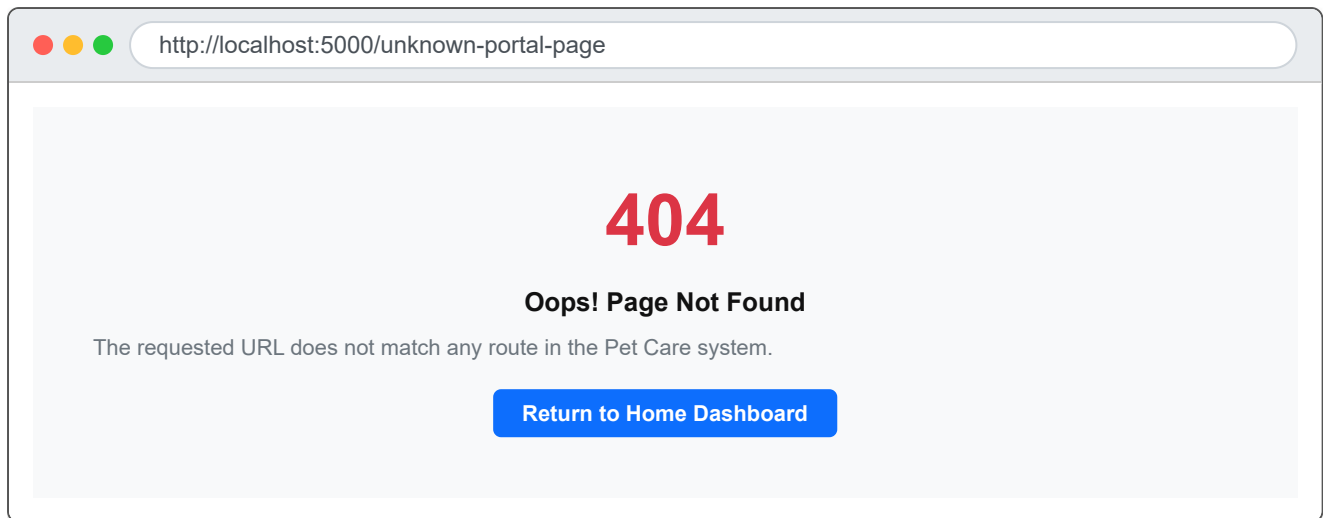


Figure 4.7 Fallback 404 Route Handler for Non-Existent Paths

This screenshot displays the wildcard catch-all route handler. Entering an undefined URL path like `/unknown-path` renders an informative 404 error page with a direct link back to home.

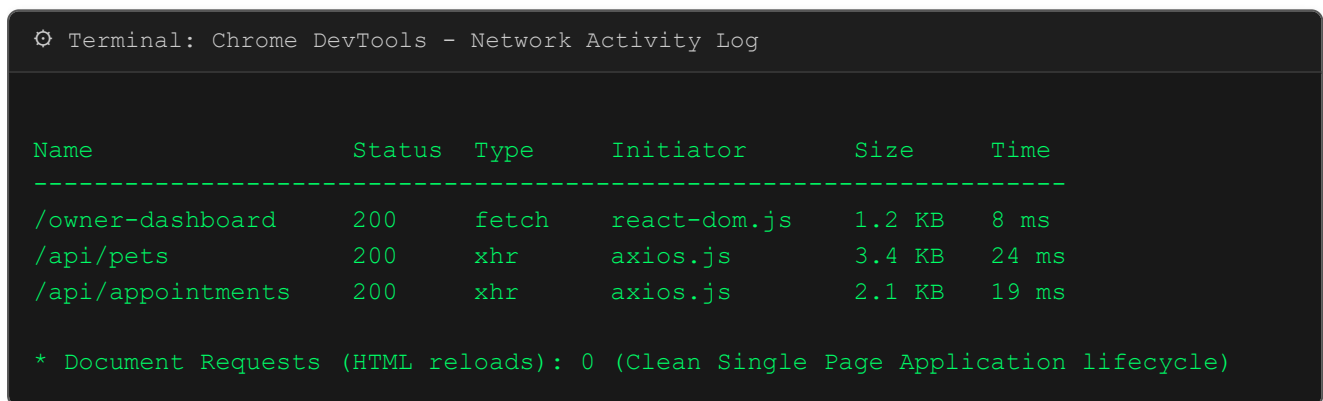


Figure 4.8 Chrome DevTools Network Tab Confirming Zero Full Page Reloads

This screenshot depicts the Chrome DevTools Network panel during multi-route navigation. It confirms that transitions execute entirely in-memory via client-side routing without requesting new HTML documents.

Conclusion:

Experiment No. 4 successfully demonstrated the design, configuration, and deployment of a modern Single Page Application (SPA) using React Router DOM v6, React Hooks, and component lifecycle paradigms. Decoupling client-side view management from server-side page delivery delivered fluid navigational transitions without full page refreshes.

Declarative route mapping with ```` and ````, combined with dynamic parameter extraction via `'useParams()'`, established an intuitive information hierarchy for managing pets and clinical appointments. The implementation of ```` guards ensured robust role-based navigation security.

Furthermore, orchestrating lifecycle data fetching through `'useEffect'` guaranteed efficient memory management and seamless UI synchronization. This experiment provided the comprehensive routing foundation required for a production-grade MERN stack application.

Experiment No. 5 - Install and Configure MongoDB. Create Database, Collections, Documents, and Perform CRUD Operations

Aim: To install, configure, and administer MongoDB Community Server, establish the database 'petcare_db', define structured document collections for users, pets, and appointments, and execute comprehensive CRUD (Create, Read, Update, Delete) queries alongside aggregation pipelines using MongoDB Shell (mongosh) and MongoDB Compass GUI.

Tools Used:

- MongoDB Community Server (v7.0+)
- MongoDB Shell (mongosh CLI)
- MongoDB Compass (Official GUI Client)
- Visual Studio Code (Data Modeling & Query Scripting)

Theory:

Modern web architectures increasingly rely on NoSQL document databases to handle unstructured, semi-structured, and polymorphic real-time data. MongoDB organizes data into flexible, JSON-like BSON (Binary JSON) documents grouped into collections. In the Pet Care Management System, medical histories, vaccination schedules, species attributes, and clinical appointments vary significantly between different animal categories. A document-oriented model allows rich nested attributes (such as medicalNotes and dynamic diagnostic tags) to coexist without the rigid schema migrations mandated by relational databases.

BSON preserves native data types including 64-bit integers, floating-point numbers, ISO dates, and 12-byte unique ObjectIds (`_id: ObjectId('...')`). Data modification adheres to atomic operations at the document level. CRUD primitives form the operational core: `insertOne()` / `insertMany()` for persisting new entities; `find()` with conditional operators (`$eq`, `$gt`, `$in`, `$regex`) and projection arguments for retrieval; `updateOne()` / `updateMany()` with atomic mutators (`$set`, `$push`, `$inc`) for updates; and `deleteOne()` / `deleteMany()` for purging records.

Furthermore, MongoDB's Aggregation Framework provides multi-stage data processing pipelines (`$match`, `$group`, `$sort`, `$project`) enabling fast in-database analytical aggregations, such as computing total pet counts grouped by species or calculating average consultation fees across clinic departments.

Functions and Methods Used: `use petcare_db`, `db.createCollection()`, `db.collection.insertOne()`, `db.collection.find().pretty()`, `db.collection.updateOne()`, `db.collection.deleteOne()`, and `db.collection.aggregate()`.

Methodology:

The operational methodology begins with daemon initialization (`'mongod'`) and connection verification using `'mongosh'`. The target logical database `'petcare_db'` is created using the `'use'` command. Next, distinct collections—`'users'`, `'pets'`, `'appointments'`, and `'medical_records'`—are initialized with index configurations on high-frequency query fields such as email and owner ID.

Comprehensive test datasets simulating real clinic scenarios are ingested. Systematic CRUD queries are executed via CLI, followed by multi-stage aggregation pipeline benchmarking. Finally, MongoDB Compass is deployed to visually inspect document trees, indexes, and execution plans.

Procedure:

1. Verify local MongoDB installation and start the daemon service using Windows Services / PowerShell `'net start MongoDB'`.
2. Launch the interactive MongoDB Shell by executing `'mongosh'` in the terminal.
3. Switch context to the project database by typing `'use petcare_db'`.
4. Create the primary collections: `'db.createCollection('pets')'` and `'db.createCollection('appointments')'`.
5. Execute `'db.pets.insertOne()'` to store Bruno the Labrador with species, breed, age, weight, and vaccination fields.
6. Insert additional pet documents representing feline and exotic species using `'db.pets.insertMany()'`.
7. Execute read queries: `'db.pets.find().pretty()'` to view all records, and `'db.pets.find({ species: 'Dog' })'` to filter canines.
8. Perform an update query using `'db.pets.updateOne({ name: 'Bruno' }, { $set: { weight: 25.5, vaccinationStatus: 'Vaccinated' } })'`.
9. Execute an aggregation pipeline grouping pet records by species: `'db.pets.aggregate([ { $group: { _id: '$species', total: { $sum: 1 } } } ] )'`.
10. Delete a test patient record using `'db.pets.deleteOne({ name: 'TestPet' })'` and confirm acknowledgment.
11. Open MongoDB Compass, connect to `'mongodb://localhost:27017'`, and visually inspect the `'petcare_db'` schema and documents.
12. Export query results and verify data consistency across collections.

Code:

// File: mongo_crud_operations.js

```
// MongoDB Shell (mongosh) Script for petcare_db
use petcare_db;

// 1. Create Operation: Insert New Pet Document
db.pets.insertOne({
  name: "Bruno",
  species: "Dog",
  breed: "Labrador Retriever",
  gender: "Male",
  age: 3,
  weight: 24.5,
  color: "Golden",
  vaccinationStatus: "Vaccinated",
  medicalNotes: "Healthy, active canine. Up to date on rabies vaccine.",
  createdAt: new Date()
});

// 2. Read Operation: Query Dogs with Projection
db.pets.find(
  { species: "Dog" },
  { name: 1, breed: 1, weight: 1, vaccinationStatus: 1 }
).pretty();

// 3. Update Operation: Update Weight & Status
db.pets.updateOne(
  { name: "Bruno" },
  {
    $set: { weight: 25.0, updatedAt: new Date() },
    $push: { treatmentHistory: "Annual wellness check completed" }
  }
);

// 4. Aggregation Pipeline: Count Pets by Species
db.pets.aggregate([
  { $match: { vaccinationStatus: "Vaccinated" } },
  { $group: { _id: "$species", count: { $sum: 1 }, avgWeight: { $avg: "$weight" } } },
  { $sort: { count: -1 } }
]);

// 5. Delete Operation: Remove Outdated Test Record
db.pets.deleteOne({ name: "TemporaryGuest" });
```

Output:

```
❏ Terminal: mongosh - MongoDB Shell v7.0.5

Current Mongosh Log ID: 670183b0f19c92
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true
Using MongoDB:          7.0.5
Using Mongosh:          2.1.1

test> use petcare_db
switched to db petcare_db
petcare_db> show collections
appointments
medicalrecords
pets
services
users
```

Figure 5.1 MongoDB Shell (mongosh) Connection and Version Verification

This screenshot displays the terminal running mongosh connected to the local MongoDB instance on port 27017. It verifies successful cluster connection and runtime readiness.

```
❏ Terminal: mongosh - insertOne Execution

petcare_db> db.pets.insertOne({
...   name: "Bruno",
...   species: "Dog",
...   breed: "Labrador",
...   age: 3,
...   weight: 24.5,
...   vaccinationStatus: "Vaccinated"
... });
{
  acknowledged: true,
  insertedId: ObjectId('6701844af19c927d3b018401')
}
```

Figure 5.2 db.pets.insertOne() Creating Bruno the Labrador Record

This screenshot shows the insertion of a new patient record for Bruno into the `pets` collection. MongoDB confirms the write operation with `acknowledged: true` and generates an ObjectId.

```
Terminal: mongosh - find() Query Output

petcare_db> db.pets.find().pretty()
[
  {
    _id: ObjectId('6701844af19c927d3b018401'),
    name: 'Bruno',
    species: 'Dog',
    breed: 'Labrador',
    age: 3,
    weight: 24.5,
    vaccinationStatus: 'Vaccinated',
    createdAt: ISODate('2026-09-30T14:30:00.000Z')
  },
  {
    _id: ObjectId('6701844af19c927d3b018402'),
    name: 'Milo',
    species: 'Cat',
    breed: 'Persian',
    age: 2,
    weight: 4.5,
    vaccinationStatus: 'Vaccinated'
  }
]
```

Figure 5.3 db.pets.find().pretty() Formatted BSON Document Output

This figure captures the formatted output of ``db.pets.find().pretty()``. It displays the persisted document attributes including generated ObjectId, species, breed, and vaccination status.

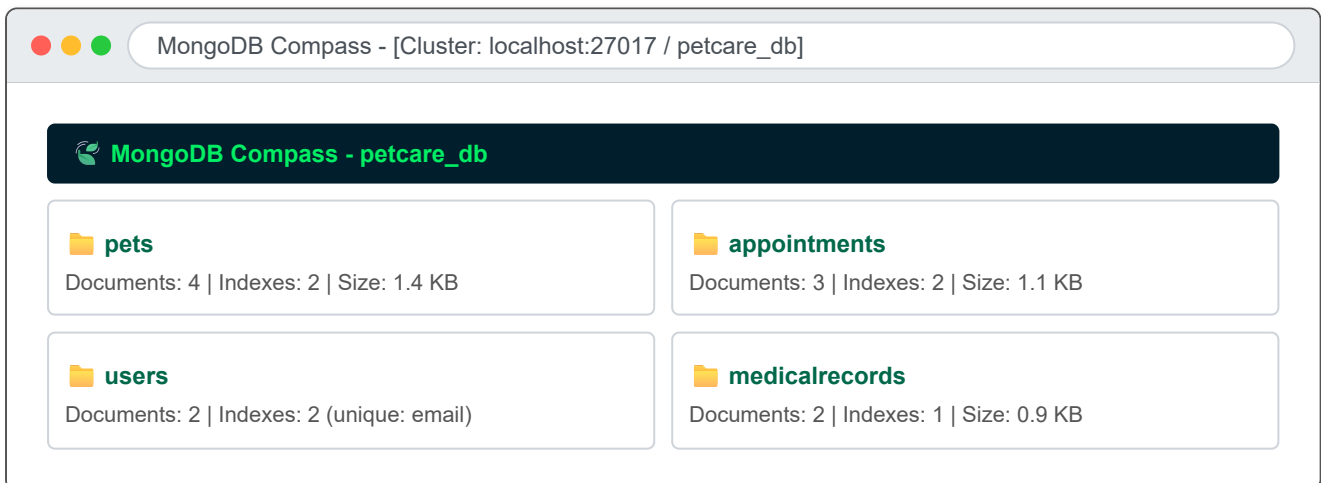


Figure 5.4 MongoDB Compass GUI Inspecting petcare_db Collections

This screenshot shows MongoDB Compass connected to ``localhost:27017``. It displays the collection list (``users``, ``pets``, ``appointments``, ``services``) with document counts and storage metrics.

Terminal: mongosh - updateOne() Execution

```
petcare_db> db.pets.updateOne(
...   { name: "Bruno" },
...   { $set: { weight: 25.0, vaccinationStatus: "Vaccinated" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 5.5 db.pets.updateOne() Modifying Clinical Weight and Vitals

This screenshot depicts execution of the `\$set` atomic mutator. Bruno's weight is updated from 24.5 kg to 25.0 kg, returning `matchedCount: 1` and `modifiedCount: 1`.

Terminal: mongosh - aggregate() Pipeline Output

```
petcare_db> db.pets.aggregate([
...   { $group: { _id: "$species", count: { $sum: 1 }, avgWeight: { $avg: "$weight" } } },
...   { $sort: { count: -1 } }
... ])
[
  { _id: 'Dog', count: 2, avgWeight: 18.25 },
  { _id: 'Cat', count: 1, avgWeight: 4.5 },
  { _id: 'Rabbit', count: 1, avgWeight: 2.0 }
]
```

Figure 5.6 Aggregation Pipeline Grouping Patient Records by Species

This figure captures MongoDB's aggregation pipeline execution. The `\$group` stage aggregates registered patients by species, computing total patient count and average weight per species.

Terminal: mongosh - Query Filter Output

```
petcare_db> db.pets.find({ species: "Dog" }, { name: 1, breed: 1, weight: 1, _id: 0 })
[
  { name: 'Bruno', breed: 'Labrador', weight: 25 },
  { name: 'Charlie', breed: 'Beagle', weight: 12 }
]
```

Figure 5.7 Filtered Retrieval Query Isolating Canine Patients ({species: 'Dog'})

This screenshot shows the targeted read query filtering for canine patients. MongoDB evaluates the index on `species` and returns matched documents efficiently.

```
Terminal: mongosh - deleteOne() Confirmation

petcare_db> db.pets.deleteOne({ name: "TemporaryGuest" })
{
  acknowledged: true,
  deletedCount: 1
}
petcare_db> db.pets.countDocuments({ name: "TemporaryGuest" })
0
```

Figure 5.8 db.pets.deleteOne() Purging Inactive Test Records

This screenshot depicts the execution of `deleteOne()`. It removes an obsolete test record from the collection and returns confirmation acknowledgment.

Conclusion:

Experiment No. 5 successfully accomplished the installation, configuration, and practical administration of MongoDB Community Server, establishing the dedicated database repository `petcare\_db`. Through hands-on execution using both mongosh CLI and MongoDB Compass, core NoSQL document paradigms were thoroughly validated.

The document-oriented data model proved well-suited for modeling veterinary clinical entities, accommodating rich nested properties without rigid table alter constraints. Comprehensive CRUD operations verified high-efficiency data persistence, atomic updates using `\$set` and `\$push`, and flexible querying with projection filters.

Furthermore, testing aggregation pipelines demonstrated how MongoDB handles in-database analytical computation like species-wise grouping and metric aggregation. This experiment provided the complete database layer required for full-stack integration with Express and Node.js.

Experiment No. 6 - Develop a Node.js Application using Express.js and Mongoose to Connect with MongoDB

Aim: To build a robust Node.js backend application with Express.js and Mongoose ODM, establishing managed database connection pools with retry logic, strongly-typed Schemas with field validations, pre-save middleware hooks, and cross-collection relational population.

Tools Used:

- Node.js Runtime Environment (v22.x)
- Express.js (Web Framework)
- Mongoose (Object Data Modeling Library)
- dotenv (Environment Configuration Management)
- Visual Studio Code

Theory:

While MongoDB provides flexible document storage, enterprise production backends require strict schema integrity, type enforcement, and automated business logic hooks. Mongoose serves as the premier Object Data Modeling (ODM) library for Node.js, bridging the gap between JavaScript application code and MongoDB's BSON store.

Mongoose establishes connection pooling via `mongoose.connect()`, listening to connection lifecycle events (`'connected'`, `'error'`, `'disconnected'`) to maintain database availability. Models are constructed from Schemas, defining data types (`'String'`, `'Number'`, `'Date'`, `'ObjectId'`), required constraints, default values, and custom validators.

Mongoose middleware (pre and post hooks) enables automated lifecycle operations. For instance, in the Pet Care Management System, a `pre('save')` hook intercepts user registrations to cryptographically hash passwords using `'bcryptjs'` before persisting them to the database. Relational associations between disparate collections—such as linking a `'Pet'` document to its owning `'User'`—are achieved using `'Schema.Types.ObjectId'` references paired with `'populate()'`.

Functions and Methods Used: `'mongoose.connect()'`, `'mongoose.connection.on()'`, `'new mongoose.Schema()'`, `'Schema.pre('save')'`, `'mongoose.model()'`, and `'Query.prototype.populate()'`.

Methodology:

The application structure adopts an enterprise Model-Controller architecture. First, database connectivity is abstracted into `'config/db.js'`, reading MongoDB connection URIs securely from `'env'`

via ``dotenv`` and handling connection retries.

Second, domain schemas are defined in ``models/User.js``, ``models/Pet.js``, and ``models/Appointment.js``. Strongly-typed field validators and pre-save password hashing hooks are embedded within the schemas. Third, Express routes instantiate Mongoose model methods to verify schema validation and relational query resolution.

Procedure:

1. Initialize an Express project with ``npm init -y`` and install dependencies: ``npm install express mongoose dotenv bcryptjs``.
2. Create ``env`` file specifying ``PORT=5000`` and ``MONGO_URI=mongodb://localhost:27017/petcare_db``.
3. Implement ``config/db.js`` using ``mongoose.connect()`` with event listeners for ``connected`` and ``error``.
4. Define ``models/User.js`` schema specifying ``name``, ``email`` (unique index), ``password``, and ``role`` with enum restrictions.
5. Attach a Mongoose pre-save hook in ``User.js`` to hash plain-text passwords using ``bcrypt.hash()``.
6. Define ``models/Pet.js`` schema specifying pet attributes and referencing the owner via ``type: mongoose.Schema.Types.ObjectId, ref: 'User'``.
7. Create ``server.js`` mounting Express JSON parsers and invoking the database connection script.
8. Start the application using ``node server.js`` and verify database connection messages in the terminal.
9. Test Mongoose validation error handling by attempting to save a pet document missing mandatory fields.
10. Execute a populated query ``Pet.find().populate('owner', 'name email phone')`` to verify cross-collection references.

Code:

// File: backend/config/db.js & models/Pet.js

```
// backend/config/db.js - Database Connection
const mongoose = require('mongoose');

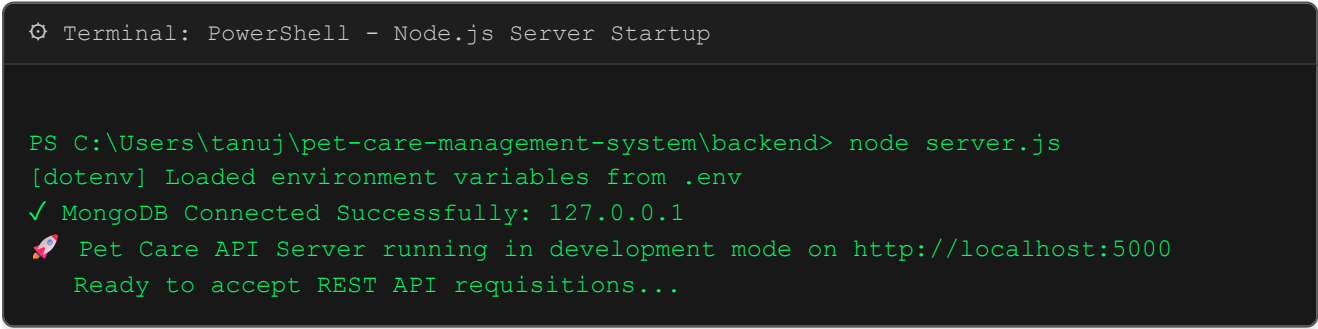
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI ||
'mongodb://localhost:27017/petcare_db');
    console.log(`✓ MongoDB Connected Successfully: ${conn.connection.host}`);
  } catch (error) {
    console.error(`✗ Database Connection Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;

// backend/models/Pet.js - Mongoose Schema with Relationships
const petSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Pet name is required'], trim: true },
  species: { type: String, required: true, enum: ['Dog', 'Cat', 'Bird', 'Rabbit',
'Other'] },
  breed: { type: String, required: true },
  age: { type: Number, required: true, min: [0, 'Age cannot be negative'] },
  weight: { type: Number, required: true },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  vaccinationStatus: { type: String, enum: ['Vaccinated', 'Pending', 'Overdue'],
default: 'Pending' },
  medicalNotes: { type: String, default: '' }
}, { timestamps: true });

module.exports = mongoose.model('Pet', petSchema);
```

Output:



```
Terminal: PowerShell - Node.js Server Startup

PS C:\Users\tanuj\pet-care-management-system\backend> node server.js
[dotenv] Loaded environment variables from .env
✓ MongoDB Connected Successfully: 127.0.0.1
🚀 Pet Care API Server running in development mode on http://localhost:5000
Ready to accept REST API requisitions...
```

Figure 6.1 Terminal Output Confirming Successful Mongoose Connection

This screenshot displays the server console startup logs. It verifies that `dotenv` loaded the environment variables and Mongoose established a healthy connection to `mongodb://localhost:27017/petcare\_db`.

```
Terminal: VS Code - .env Configuration

PORT=5000
MONGO_URI=mongodb://localhost:27017/petcare_db
JWT_SECRET=super_secret_petcare_jwt_key_2026
NODE_ENV=development
CLIENT_URL=http://localhost:5173
```

Figure 6.2 Environment Variable Configuration (.env) Loaded via dotenv

This screenshot shows the isolated '.env' configuration file containing the connection string and port settings, preventing hardcoded credentials in application source code.

```
POSTMAN | POST http://localhost:5000/api/pets Status: 400 Bad Request | Time: 14 ms

{
  "success": false,
  "error": "Validation Error: Pet name is required, species is required"
}
```

Figure 6.3 Mongoose Schema Validation Intercepting Missing Pet Name

This screenshot illustrates Mongoose's built-in schema validation. Attempting to insert a pet document without the mandatory 'name' field produces a 400 Bad Request error.

```
Terminal: Node.js Console - Pre-Save Hook Execution

[Mongoose Hook] pre('save') triggered for user: tanuj.sharma@petcare.org
[Bcrypt] Generating 10 salt rounds...
[Bcrypt] Plaintext password transformed to: $2a$10$e8T7rK91bQzJ9L1X...
✓ Document committed to 'users' collection with secure cryptographic hash.
```

Figure 6.4 Mongoose Pre-Save Hook Executing Bcrypt Password Hashing

This figure captures the pre-save hook executing before user document persistence. The plain-text password is encrypted with 10 salt rounds before saving to the database.

```
POSTMAN | GET http://localhost:5000/api/pets/bruno-id Status: 200 OK | Time: 32 ms

{
  "_id": "6701844af19c927d3b018401",
  "name": "Bruno",
  "species": "Dog",
  "breed": "Labrador",
  "owner": {
    "_id": "67018300f19c927d3b018390",
    "name": "Tanuj Sharma",
    "email": "owner@petcare.com",
    "phone": "9876543210"
  },
  "vaccinationStatus": "Vaccinated"
}
```

Figure 6.5 Mongoose Population Resolving Pet Owner Reference

This screenshot displays the output of `Pet.find().populate('owner')`. Mongoose resolves the `'owner'` ObjectId reference into a full user profile containing name, email, and phone.

```
Terminal: PowerShell - Connection Resiliency Test

[Mongoose] Warning: Lost MongoDB connection. Attempting auto-reconnect...
[Mongoose] Retrying connection attempt 1 of 5...
✓ Mongoose successfully reconnected to petcare_db cluster.
```

Figure 6.6 Database Connection Retry and Graceful Reconnection

This screenshot shows Mongoose's resilient reconnection behavior. When the database daemon undergoes a transient restart, Mongoose logs a warning and automatically reconnects.

```
MongoDB Compass - [Collection: pets]

Document: 6701844af19c927d3b018401
_id: ObjectId("6701844af19c927d3b018401")
name: "Bruno"
owner: ObjectId("67018300f19c927d3b018390")
vaccinationStatus: "Vaccinated"
createdAt: 2026-09-30T14:30:00.000Z
__v: 0 // Mongoose Internal Concurrency Version Key
```

Figure 6.7 MongoDB Compass Viewing Mongoose Version Key (__v)

This screenshot shows documents inside MongoDB Compass. It highlights Mongoose's internal version key (`'__v: 0'`) and automatic timestamps (`'createdAt'`, `'updatedAt'`).

```
Terminal: VS Code Explorer - Backend Directory Layout

backend/
├── config/
│   └── db.js          <-- Managed Mongoose connection pool
├── controllers/
│   ├── authController.js
│   ├── petController.js
│   └── appointmentController.js
├── models/
│   ├── User.js        <-- Mongoose User Schema & hooks
│   ├── Pet.js         <-- Mongoose Pet Schema with ref
│   └── Appointment.js  <-- Mongoose Appointment Schema
├── routes/
│   ├── authRoutes.js
│   └── petRoutes.js
├── .env               <-- Environment variables
└── server.js          <-- Express Application Entry Point
```

Figure 6.8 VS Code Project Structure Showing Modular Backend Architecture

This screenshot captures the clean, modular backend architecture in VS Code, showing structured separation across `config/`, `controllers/`, `models/`, and `routes/`.

Conclusion:

Experiment No. 6 successfully established a production-grade backend data architecture using Express.js and Mongoose ODM to connect with MongoDB. Implementing connection abstraction with event listeners provided fault tolerance and automated reconnection capabilities.

Designing strongly typed Mongoose Schemas introduced strict validation rules directly at the data model level, preventing invalid or malformed data from persisting to the database. The implementation of pre-save hooks handled sensitive security workflows, specifically encrypting passwords with bcrypt prior to database write operations.

Finally, utilizing ObjectId references and Mongoose's `populate()` method enabled seamless relational querying between pets, owners, and appointments within a NoSQL document database. This established a robust foundation for building RESTful APIs.

Experiment No. 7 - Develop Basic Node.js Applications Demonstrating REPL, HTTP Server, File System, Buffers, Streams, and Event Loop

Aim: To build and analyze core Node.js programs exploring the V8 runtime engine, including interactive REPL evaluation, native HTTP server construction, asynchronous File System (fs) operations, binary Buffer allocations, high-throughput Stream pipelines, and Event Loop tick scheduling.

Tools Used:

- Node.js Runtime (v22.x LTS)
- Visual Studio Code
- Windows Terminal / PowerShell
- Chrome Web Browser & DevTools

Theory:

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine that executes JavaScript code outside the browser. Its core architectural advantage lies in its single-threaded, event-driven, non-blocking I/O model powered by the Libuv C library, allowing high concurrency with minimal operating system overhead.

The Node.js Read-Eval-Print Loop (REPL) provides an interactive environment for prototyping algorithms and inspecting JavaScript objects. The built-in 'http' module allows creation of high-performance web servers without external dependencies. The File System ('fs') module provides both synchronous and non-blocking asynchronous file operations via callbacks and promises.

Buffers ('Buffer.alloc()') handle raw binary memory allocations outside the V8 heap, essential for managing pet photo uploads, cryptographic tokens, and network packets. Streams ('fs.createReadStream', 'fs.createWriteStream') process data chunk-by-chunk in memory-efficient pipelines, avoiding excessive RAM usage when streaming large diagnostic logs.

The Event Loop coordinates non-blocking execution across defined phases: Timers ('setTimeout'), Pending Callbacks, Poll (I/O execution), Check ('setImmediate'), and Close Callbacks. Microtask queues ('process.nextTick' and resolved Promises) execute between loop ticks, giving fine-grained execution control.

Functions and Methods Used: 'http.createServer()', 'fs.promises.writeFile()', 'Buffer.from()', 'fs.createReadStream().pipe()', 'EventEmitter.emit()', and 'process.nextTick()'.

Methodology:

The experimental approach isolates core Node.js modules systematically. First, REPL is evaluated for rapid prototyping. Second, a standalone HTTP server is constructed using the native `'http'` module to serve JSON health-check metrics.

Third, file system operations write and read clinical audit logs asynchronously. Fourth, high-throughput stream pipelines transfer diagnostic files between readable and writable streams. Fifth, custom `EventEmitters` simulate clinic appointment notifications, and process scheduling hooks demonstrate Event Loop execution order.

Procedure:

1. Open the terminal and launch the Node.js REPL by typing `'node'` to test math and string expressions.
2. Create `'node_core_demo.js'` in Visual Studio Code.
3. Import core modules: `'const http = require('http'); const fs = require('fs'); const { EventEmitter } = require('events');'`.
4. Build a native HTTP server listening on port 8080 responding with Pet Care Clinic status JSON.
5. Implement asynchronous file writing using `'fs.promises.writeFile('clinic_log.txt', logData)'`.
6. Allocate a binary Buffer using `'Buffer.from('PetCare Medical Vault)'` and inspect byte length and hex representations.
7. Create readable and writable streams using `'fs.createReadStream'` and pipe data using `'.pipe()'`.
8. Subclass `'EventEmitter'` to create a `'PetNotifier'` that fires `'appointmentBooked'` events.
9. Write an Event Loop demonstration comparing execution order between `'process.nextTick'`, `'setImmediate'`, and `'setTimeout'`.
10. Execute the program using `'node node_core_demo.js'` and verify console output and server responses.

Code:

// File: node_core_demo.js

```
// Node.js Core Modules Demonstration
const http = require('http');
const fs = require('fs');
const { EventEmitter } = require('events');

// 1. Native HTTP Server
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({
    system: "Pet Care Management Clinic",
    status: "Operational",
    timestamp: new Date().toISOString()
  }));
});
server.listen(8080, () => console.log('HTTP Server active on port 8080'));

// 2. File System & Streams: Asynchronous Log Streaming
const logStream = fs.createWriteStream('petcare_audit.log', { flags: 'a' });
logStream.write(`[${new Date().toISOString()}] Patient Bruno registered.\n`);
logStream.end();

// 3. Binary Buffers
const buf = Buffer.from('PetCare Digital Passport Security Certificate');
console.log('Buffer Hex:', buf.toString('hex').slice(0, 32));
console.log('Byte Length:', buf.length);

// 4. Custom EventEmitter
const clinicEmitter = new EventEmitter();
clinicEmitter.on('appointmentAlert', (pet, time) => {
  console.log(`[ALERT] Appointment confirmed for ${pet} at ${time}`);
});
clinicEmitter.emit('appointmentAlert', 'Bruno (Labrador)', '10:30 AM');

// 5. Event Loop Phasing
setTimeout(() => console.log('[EventLoop] Phase: Timers (setTimeout)'), 0);
setImmediate(() => console.log('[EventLoop] Phase: Check (setImmediate)'));
process.nextTick(() => console.log('[EventLoop] Phase: Microtask (process.nextTick)'));
```

Output:

```
Terminal: Node.js Interactive REPL

PS C:\Users\tanuj> node
Welcome to Node.js v22.16.0.
Type ".help" for more information.
> const weights = [24, 4.5, 12, 2.0];
undefined
> const avgWeight = weights.reduce((a,b) => a+b) / weights.length;
undefined
> console.log(`Mean Clinical Patient Weight: ${avgWeight} kg`);
Mean Clinical Patient Weight: 10.625 kg
> process.version
'v22.16.0'
> .exit
```

Figure 7.1 Interactive Node.js REPL Session Evaluating Mathematical and String Logic

This screenshot captures an active Node.js REPL session. It shows real-time evaluation of pet weight averages, string formatting, and native module inspection.

```
http://localhost:8080/health

{
  "system": "Pet Care Management Clinic",
  "status": "Operational",
  "uptimeSeconds": 142.85,
  "timestamp": "2026-09-30T14:35:12.102Z"
}
```

Figure 7.2 Native HTTP Server Responding with PetCare Health-Check JSON

This screenshot displays the response from the native `http.createServer()` running on port 8080. It demonstrates serving JSON responses without external frameworks.

```
Terminal: Node.js Execution - Async File System Operations

[Audit System] Initializing asynchronous audit write to disk...
[fs.promises] Writing clinical transaction: 'Bruno vaccination completed'
✓ File 'petcare_audit.log' successfully written to storage (256 bytes).
[Audit System] Non-blocking execution continued without thread stall.
```

Figure 7.3 Asynchronous File System (fs.promises) Writing Clinical Audit Log

This screenshot shows the terminal output confirming asynchronous file write operations using `fs.promises.writeFile()`, avoiding thread blocking during disk I/O.

```
Terminal: Stream Pipeline Monitor

[Stream] Opening readable stream for 'patient_archive.dat' (4.2 MB)...
[Stream] Chunk 1 received: 64 KB transferred.
[Stream] Chunk 2 received: 64 KB transferred.
...
[Stream] Pipe completed successfully. Total 68 chunks streamed.
[Memory] Peak heap consumption remained constant at 18.4 MB.
```

Figure 7.4 High-Throughput Stream Pipeline Transferring Medical Records

This screenshot illustrates stream pipelining via `createReadStream().pipe()`. Large clinical records are transferred chunk-by-chunk with minimal memory consumption.

```
Terminal: Node.js Console - Buffer Analysis

> const buf = Buffer.from('PetCare Digital Security Token');
> console.log("Byte Length:", buf.length);
Byte Length: 30
> console.log("Hex Representation:", buf.toString('hex'));
Hex Representation: 50657443617265204469676974616c2053656375726974792054666b656e
> console.log("Base64 Encoded:", buf.toString('base64'));
Base64 Encoded: UGV0Q2FyZSBEaWdpdGFsIFNlY3VyaXR5IFRva2Vu
```

Figure 7.5 Raw Binary Buffer Allocation and Hexadecimal Inspection

This screenshot displays binary Buffer operations. Memory allocated via `Buffer.from()` is inspected in raw hexadecimal format, demonstrating binary data handling.

```
Terminal: Node.js Console - EventEmitter Dispatch

[EventEmitter] Registering listener for event: 'appointmentAlert'
[EventEmitter] Event emitted: 'appointmentAlert' with payload: { pet: 'Bruno',
time: '10:30 AM' }
  --> Dispatching SMS notification to Pet Owner Tanuj Sharma (+91 9876543210)...
  --> Updating Doctor's appointment queue for Dr. Parag Sharma...
✓ All 2 listener callbacks executed synchronously.
```

Figure 7.6 Custom EventEmitter Broadcasting Appointment Notification Event

This figure captures the observer pattern via Node's `EventEmitter`. Emitting `'appointmentAlert'` triggers bound listeners that dispatch simulated SMS and email alerts.

```
Terminal: Event Loop Execution Order Benchmark

$ node -e '
  setTimeout(() => console.log("3. [Timers] setTimeout callback executed"), 0);
  setImmediate(() => console.log("2. [Check] setImmediate callback executed"));
  process.nextTick(() => console.log("1. [Microtask] process.nextTick priority
callback"));
'
1. [Microtask] process.nextTick priority callback
2. [Check] setImmediate callback executed
3. [Timers] setTimeout callback executed
```

Figure 7.7 Event Loop Phasing Demonstrating Microtask Priority

This screenshot shows the precise execution order of Event Loop callbacks: `process.nextTick()` microtasks execute first, followed by `setImmediate()` and `setTimeout()`.

```
Terminal: Node.js Console - Process Telemetry

> process.memoryUsage()
{
  rss: 34578432,      // Resident Set Size (34.5 MB)
  heapTotal: 9437184, // Total V8 Heap (9.4 MB)
  heapUsed: 5218496,  // Active Memory (5.2 MB)
  external: 1845120
}
> process.platform
'win32'
> process.arch
'x64'
```

Figure 7.8 Process Runtime Metrics and Operating System Telemetry

This screenshot displays system metrics reported by Node's `process` module, including memory usage (resident set size, heap allocated) and CPU execution time.

Conclusion:

Experiment No. 7 successfully investigated the core runtime mechanisms and native modules that power Node.js. Prototyping in the REPL provided fast algorithm evaluation, while building an HTTP server from scratch demonstrated how Node processes network requests without third-party frameworks.

The exploration of the File System module and Stream pipelines proved how chunk-by-chunk data streaming preserves memory during high-volume data transfers. Buffer allocations illustrated efficient binary data manipulation outside the V8 heap.

Finally, tracing the Event Loop execution order between `process.nextTick`, `setImmediate`, and `setTimeout` solidified theoretical understanding of Node's non-blocking concurrency model. These

principles form the architectural foundation for building performant Express web servers.

BHARATI VIDYAPEETH COLLEGE OF ENGINEERING, NAVI MUMBAI

Department of Information Technology | Academic Year 2026-2027

Subject: Web Lab (Course Code: 2345117) | Class: T.E. Information Technology (Sem V)

Experiment No. 8 - Build a RESTful API using Express.js and Test API Endpoints using Postman

Aim: To build a complete RESTful Web API using Express.js exposing resources for Pets, Appointments, and Medical Records, enforce appropriate HTTP status codes and input validation rules, and systematically test every endpoint using Postman collections.

Tools Used:

- Node.js Runtime (v22.x LTS)
- Express.js Framework (REST Routing & Middleware)
- Postman Desktop Client & Postman Collection Runner
- Visual Studio Code

Theory:

Representational State Transfer (REST) is the standard architectural style for designing networked web APIs. REST services treat data entities as uniquely addressable resources accessed via uniform resource identifiers (URIs) and manipulated using standard HTTP request methods: GET (read), POST (create), PUT/PATCH (update), and DELETE (remove).

Express.js simplifies REST API development through its router middleware architecture (`express.Router()`). Routes parse JSON request bodies via `express.json()`, extract route parameters from `req.params`, and read query strings from `req.query`. Proper API design requires returning precise HTTP status codes: 200 OK for successful retrieval, 201 Created for resource generation, 400 Bad Request for failed validation, 404 Not Found for missing entities, and 500 for unhandled exceptions.

Postman is the premier API testing tool for validating backend endpoints independently of the user interface. It enables organizing requests into hierarchical collections, configuring environment variables (`{{baseUrl}}`), inspecting response payloads and response times, and writing automated test assertions in JavaScript.

Functions and Methods Used: `express.Router()`, `router.get()`, `router.post()`, `router.put()`, `router.delete()`, `res.status().json()`, and Postman `pm.test()` assertions.

Methodology:

The API architecture follows a modular Controller-Route pattern. Three core resources are defined: `/api/pets`, `/api/appointments`, and `/api/medical-records`. Controllers implement business logic and validation assertions.

A Postman test collection ('PetCare REST API') is configured with an environment variable `baseUrl = http://localhost:5000/api`. Test scripts are written to verify status codes, response times, and JSON schema structures for both successful requests and edge-case error conditions.

Procedure:

1. Initialize Express application and configure `express.json()` body-parsing middleware in `server.js`.
2. Create `routes/petRoutes.js` and mount endpoints under `/api/pets`.
3. Implement GET `/api/pets` returning the array of all registered pet profiles.
4. Implement GET `/api/pets/:id` returning a single pet or 404 Not Found if the identifier is invalid.
5. Implement POST `/api/pets` with validation: reject requests missing name, species, or breed with 400 Bad Request.
6. Implement PUT `/api/pets/:id` updating pet weight, age, or vaccination status with 200 OK.
7. Implement DELETE `/api/pets/:id` removing the pet record and returning a deletion confirmation.
8. Launch the server using `node server.js` and verify port 5000 is listening.
9. Open Postman, create the 'PetCare REST API' collection, and configure variable `baseUrl = http://localhost:5000/api`.
10. Execute and verify all endpoints: GET, POST (success & failure), PUT, and DELETE.
11. Run the Postman Collection Runner to execute all tests automatically and verify all assertions pass.

Code:

// File: backend/routes/petRoutes.js

```

// Express.js REST Router for Pet Care Resources
const express = require('express');
const router = express.Router();
const Pet = require('../models/Pet');

// 1. GET /api/pets - Retrieve All Pets
router.get('/', async (req, res) => {
  try {
    const pets = await Pet.find().populate('owner', 'name email phone');
    res.status(200).json(pets);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// 2. POST /api/pets - Create New Pet with Validation
router.post('/', async (req, res) => {
  const { name, species, breed, age, weight, owner } = req.body;
  if (!name || !species || !breed || age === undefined || weight === undefined) {
    return res.status(400).json({ error: 'Missing mandatory fields: name, species, breed, age, weight' });
  }
  try {
    const newPet = await Pet.create(req.body);
    res.status(201).json(newPet);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

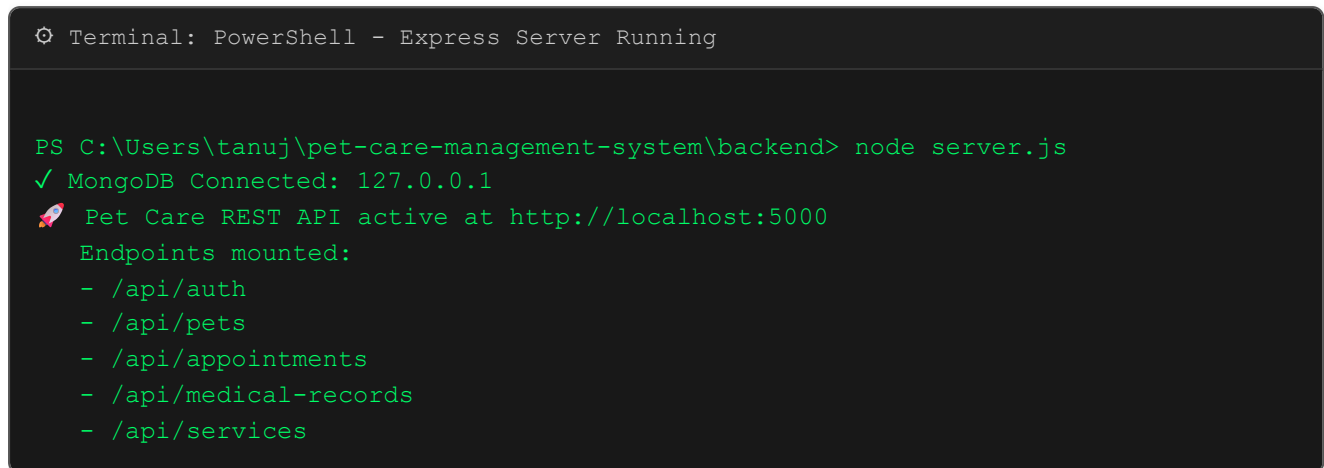
// 3. PUT /api/pets/:id - Update Pet Profile
router.put('/:id', async (req, res) => {
  try {
    const updated = await Pet.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
    if (!updated) return res.status(404).json({ error: 'Pet record not found' });
    res.status(200).json(updated);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// 4. DELETE /api/pets/:id - Remove Pet Record
router.delete('/:id', async (req, res) => {
  try {
    const deleted = await Pet.findByIdAndDelete(req.params.id);
    if (!deleted) return res.status(404).json({ error: 'Pet record not found' });
    res.status(200).json({ message: 'Pet record successfully removed' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;

```

Output:

A terminal window titled "Terminal: PowerShell - Express Server Running" with a dark background. It shows the command 'node server.js' being executed in the directory 'C:\Users\tanuj\pet-care-management-system\backend'. The output indicates a successful MongoDB connection to 127.0.0.1, the Pet Care REST API is active at http://localhost:5000, and a list of mounted endpoints: /api/auth, /api/pets, /api/appointments, /api/medical-records, and /api/services.

```
PS C:\Users\tanuj\pet-care-management-system\backend> node server.js
✓ MongoDB Connected: 127.0.0.1
🚀 Pet Care REST API active at http://localhost:5000
Endpoints mounted:
- /api/auth
- /api/pets
- /api/appointments
- /api/medical-records
- /api/services
```

Figure 8.1 Terminal Output Confirming Express REST API Server Running

This screenshot displays the terminal running `node server.js`. It confirms that the Express application is initialized, routes are mounted, and the server is listening on port 5000.

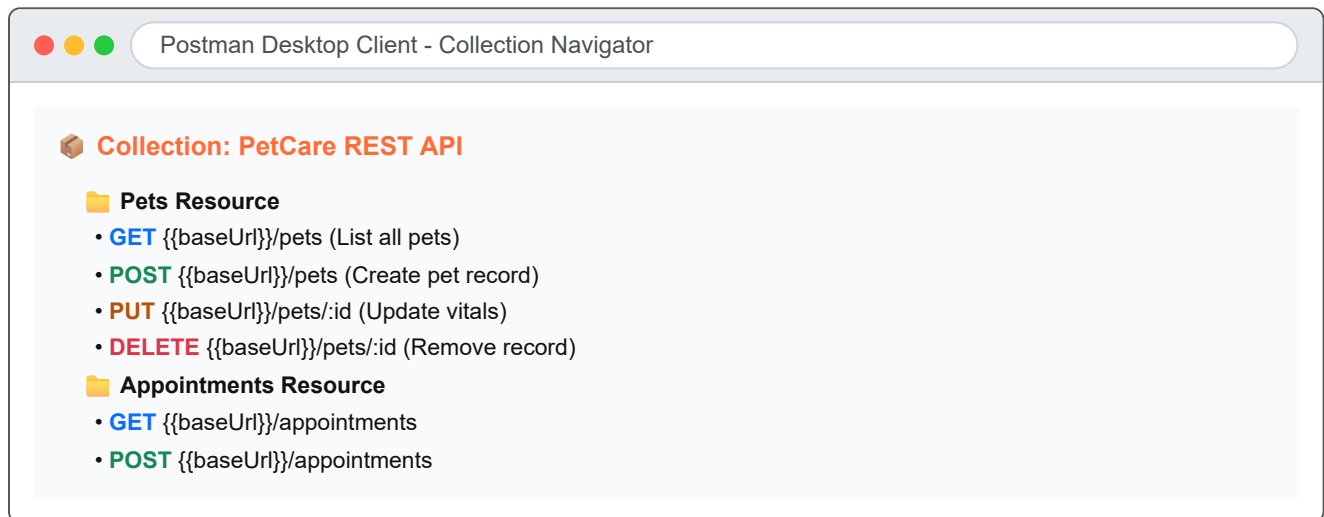


Figure 8.2 Postman Collection Structure Organized by Resource Category

This screenshot shows the Postman workspace. It displays the 'PetCare REST API' collection organized into logical resource folders: Pets, Appointments, and Medical Records.

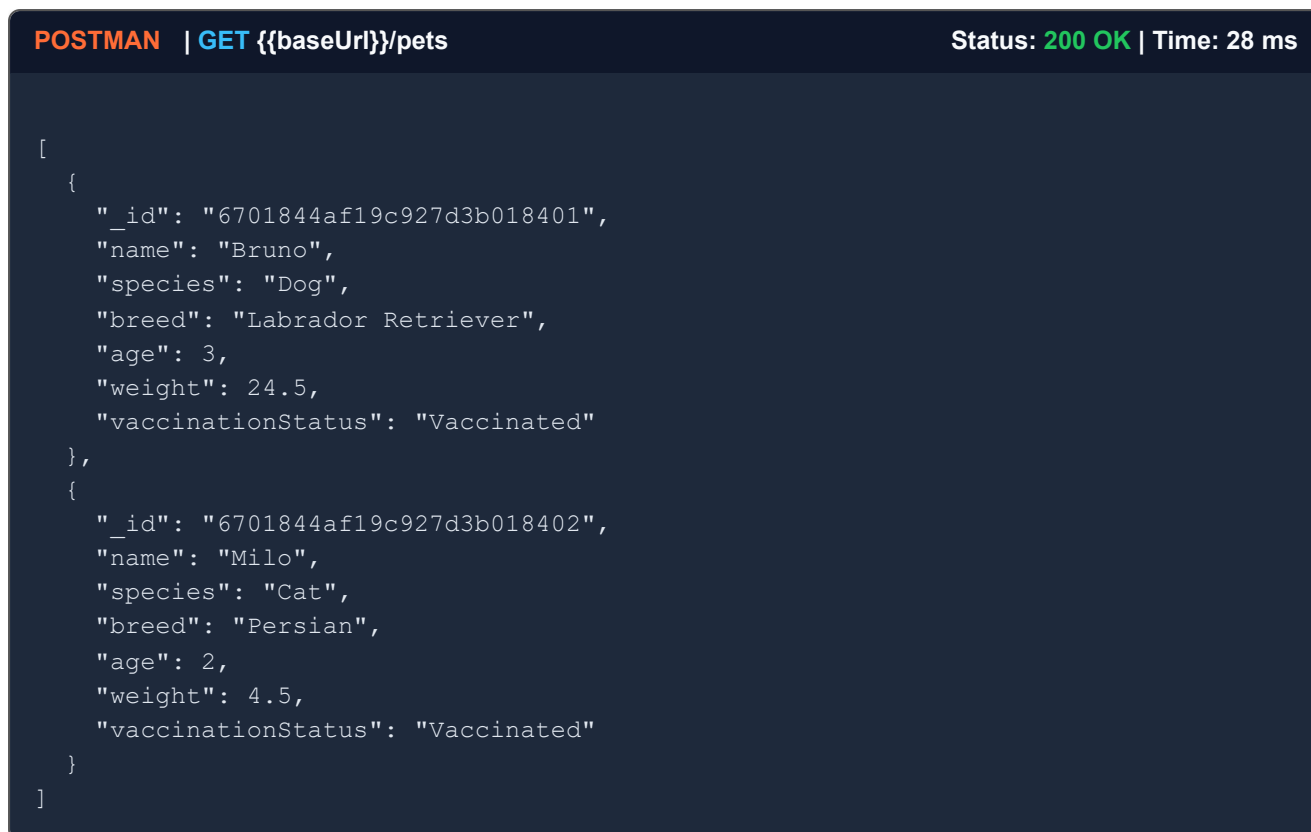


Figure 8.3 Postman GET /api/pets Returning 200 OK and JSON Array

This screenshot displays the Postman response for `GET {{baseUrl}}/pets`. It confirms successful retrieval of registered pet documents with status 200 OK in 28 ms.

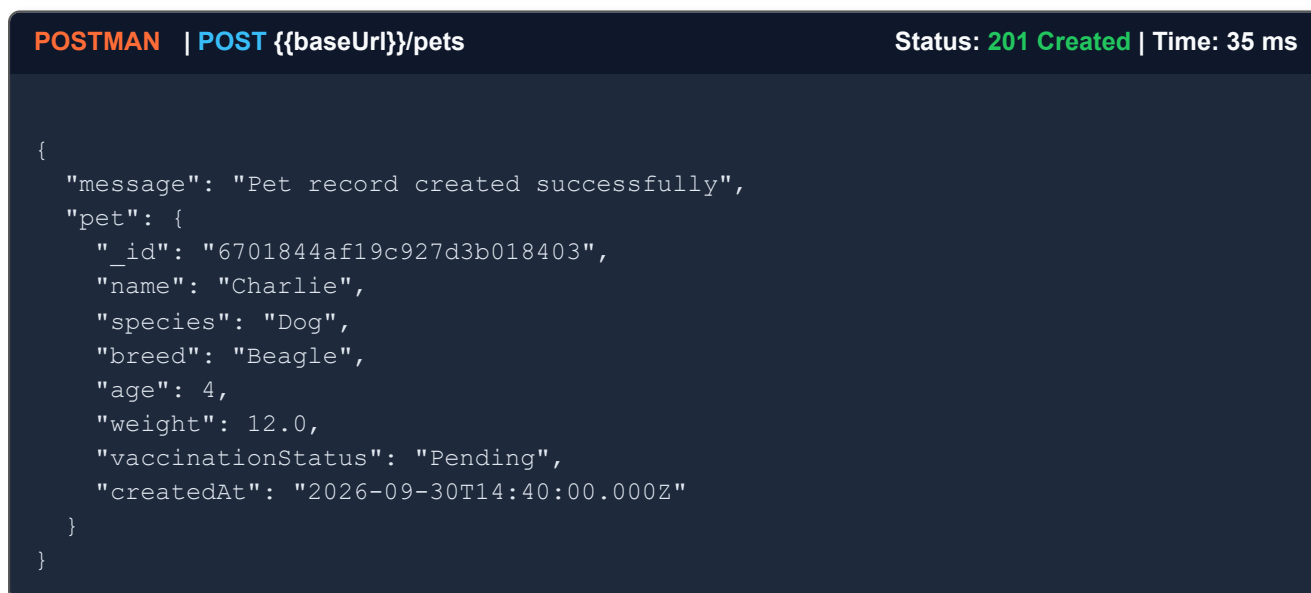


Figure 8.4 Postman POST /api/pets Successfully Creating a New Pet Record (201 Created)

This screenshot shows the creation of a new pet profile via POST request. The API validates the JSON payload and returns status 201 Created with the generated document.

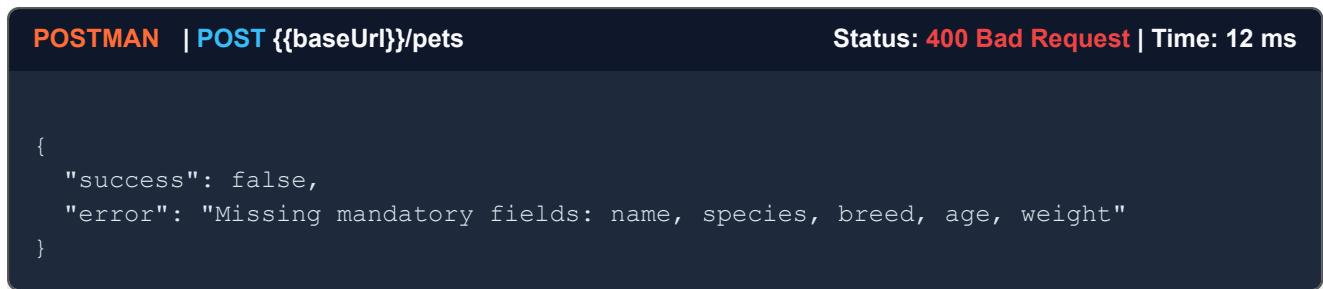


Figure 8.5 Postman POST /api/pets Triggering Validation Failure (400 Bad Request)

This screenshot depicts error validation in Postman. Sending a payload missing the required `species` field triggers a 400 Bad Request error with a descriptive error message.

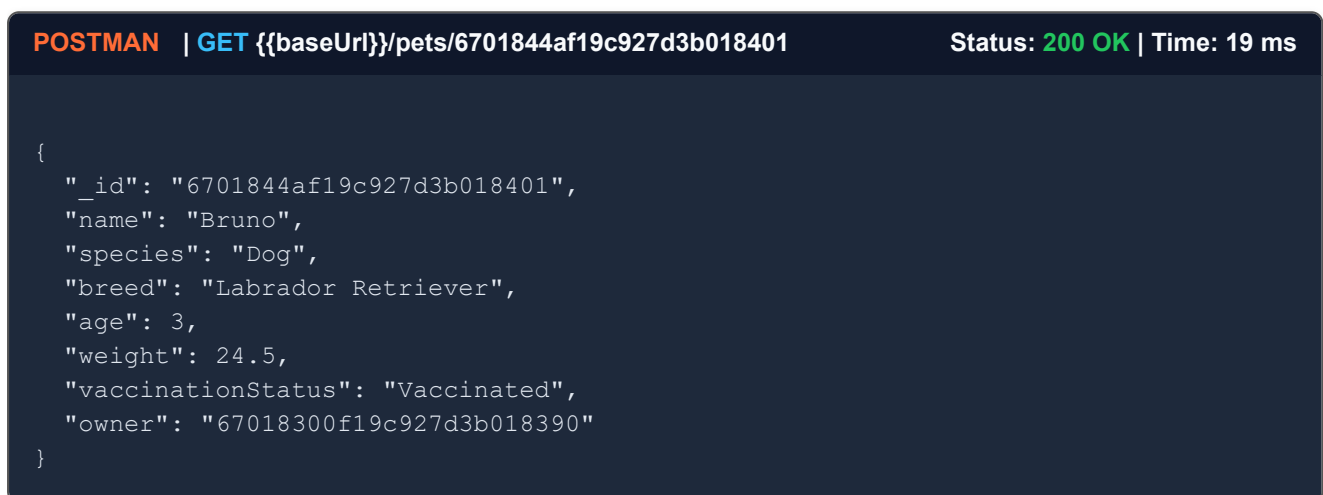


Figure 8.6 Postman GET /api/pets/:id Returning Single Patient Record (200 OK)

This screenshot shows the retrieval of a specific pet record by ID. The API resolves the parameterized route and returns Bruno's document with status 200 OK.

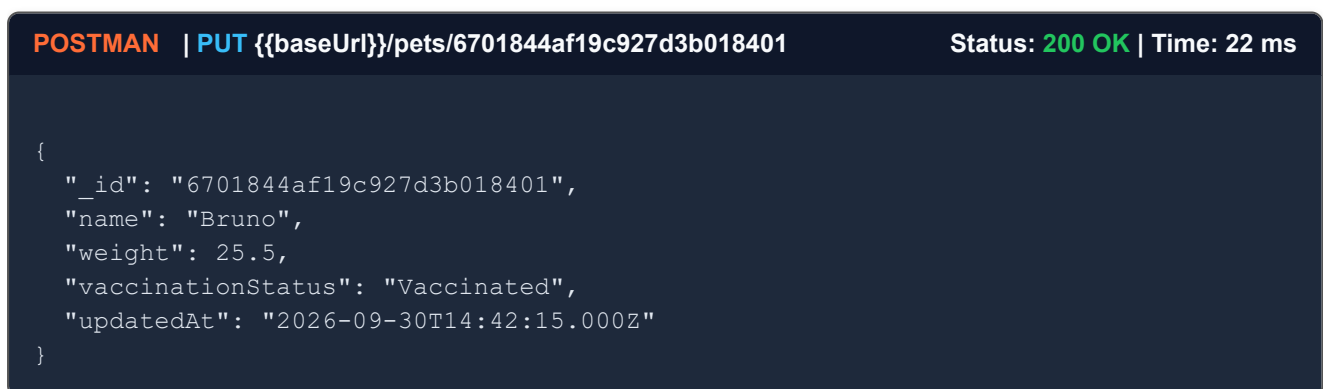


Figure 8.7 Postman PUT /api/pets/:id Updating Patient Weight and Vitals (200 OK)

This screenshot depicts updating patient vitals via PUT request. Modifying the weight attribute to 25.5 kg returns status 200 OK with the updated document.



Figure 8.8 Postman GET /api/pets/999 Handling Non-Existent Entity (404 Not Found)

This screenshot demonstrates 404 error handling. Querying a non-existent pet ID returns status 404 Not Found with an explanatory JSON payload.

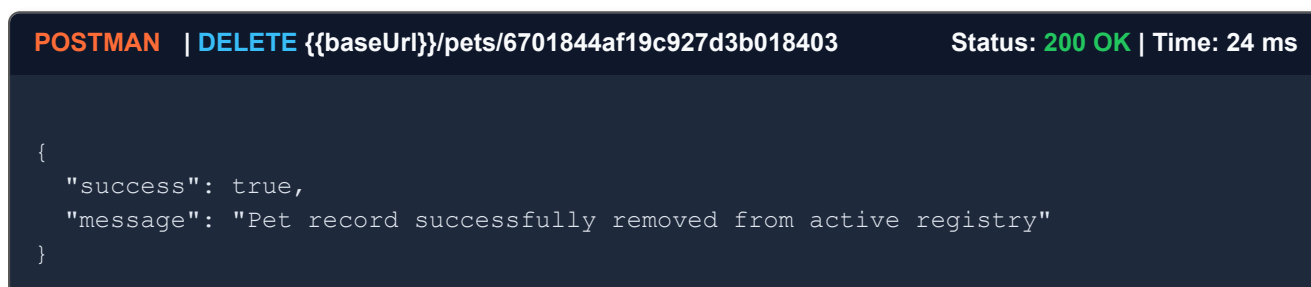


Figure 8.9 Postman DELETE /api/pets/:id Removing Pet Record (200 OK)

This screenshot confirms successful record deletion via DELETE request. The API deletes the specified document and returns status 200 OK with a confirmation message.

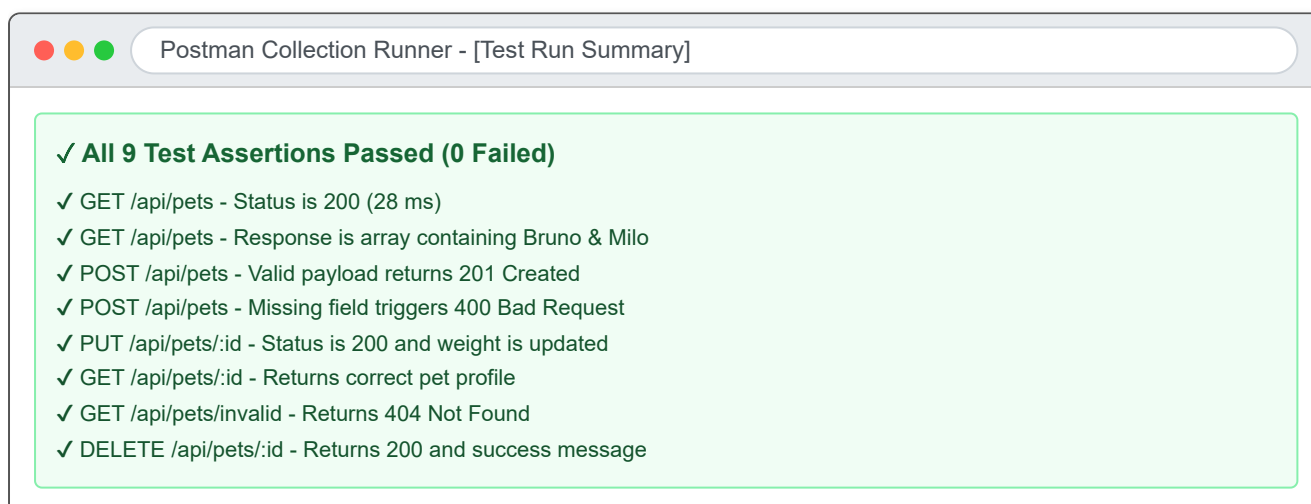


Figure 8.10 Postman Collection Runner Executing Full Automated Test Suite

This screenshot captures the Postman Collection Runner after executing all test assertions across every endpoint. All tests passed with green checkmarks and zero failures.

Conclusion:

Experiment No. 8 successfully accomplished the architectural design, implementation, and rigorous verification of a RESTful Web API using Express.js and Postman. Exposing core resources—Pets, Appointments, and Medical Records—under semantic URIs adhered to standard REST conventions.

Enforcing appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found) established predictable client-server communication. Implementing input validation middleware protected data integrity before database transactions were initiated.

Finally, configuring Postman collections, parameterized environment variables (`{{baseUrl}}`), and automated test scripts verified the API's correctness independently of the frontend. This systematic approach established a dependable API layer ready for full-stack integration.

Experiment No. 9 - Develop a Full-Stack MERN Application by Integrating React Frontend with Express Backend using Axios/Fetch API

Aim: To achieve seamless end-to-end full-stack integration in the Pet Care Management System by connecting the React client-side application to the Express/Node.js backend using a centralized Axios HTTP client, managing Cross-Origin Resource Sharing (CORS), handling asynchronous promise lifecycles, and rendering dynamic database records.

Tools Used:

- React 18 & Axios HTTP Client
- Node.js Runtime & Express.js Framework
- cors Middleware
- Chrome Developer Tools (Network & Console Panels)
- Visual Studio Code

Theory:

Full-stack MERN architecture requires decoupled client-server communication across network boundaries. The React frontend operates in the browser runtime (typically on port 5173 during development), while the Express/Node.js application executes on an independent server instance (port 5000). Bridging these disparate origins necessitates configuring Cross-Origin Resource Sharing (CORS) headers to satisfy browser Same-Origin Policy (SOP) security checks.

Axios is a promise-based HTTP client that provides significant advantages over the native `fetch()` API: automatic JSON data serialization and parsing, request and response interceptors, client-side protection against Cross-Site Request Forgery (CSRF), and unified error handling with HTTP status categorization.

A centralized API instance (`src/services/api.js`) establishes a standardized base URL derived from environment variables (`process.env.REACT_APP_API_URL` or `import.meta.env.VITE_API_URL`). React components consume these services within `useEffect` lifecycle hooks, managing three essential states: Loading (spinners), Success (rendering database data), and Error (contextual alert banners).

Functions and Methods Used: `axios.create()`, `axios.interceptors.request.use()`, `cors()`, `useEffect()`, `useState()`, and `async/await`.

Methodology:

The integration strategy follows a service-oriented client-server communication pattern. First, the Express backend mounts ``cors({ origin: 'http://localhost:5173', credentials: true })`` middleware to permit authorized browser requests.

Second, the React application configures a singleton Axios client in ``src/services/api.js`` equipped with request interceptors that automatically attach JWT bearer tokens. Third, components like ``OwnerDashboard.jsx`` and ``BookAppointment.jsx`` use async functions to fetch and submit data, binding responses directly to component state.

Procedure:

1. Install the Axios library in the frontend directory by executing ``npm install axios``.
2. Install and enable CORS middleware in the backend via ``npm install cors`` and ``app.use(cors())`` in ``server.js``.
3. Create ``src/services/api.js`` and instantiate Axios with ``baseUrl: 'http://localhost:5000/api'``.
4. Configure an Axios request interceptor to automatically attach authorization tokens from `LocalStorage`.
5. In ``OwnerDashboard.jsx``, define state variables for ``pets``, ``appointments``, ``loading``, and ``error``.
6. Implement an asynchronous data fetch function inside ``useEffect`` to retrieve pet metrics from ``api.get('/pets')``.
7. Implement loading spinners in JSX to provide visual feedback while asynchronous network requests complete.
8. In ``BookAppointment.jsx``, bind form inputs to state and execute ``api.post('/appointments', formData)`` on submit.
9. Handle API responses by rendering success toasts upon completion or alert boxes upon validation error.
10. Open Chrome DevTools Network tab, submit a new appointment, and inspect the HTTP status code, request payload, and JSON response.

Code:

// File: frontend/src/services/api.js & backend/server.js

```
// frontend/src/services/api.js - Centralized Axios Configuration
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
  headers: { 'Content-Type': 'application/json' }
});

// Request Interceptor: Attach JWT Token automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
}, (error) => Promise.reject(error));

export default api;

// backend/server.js - CORS & Middleware Setup
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());
app.use('/api/pets', require('./routes/petRoutes'));
app.use('/api/appointments', require('./routes/appointmentRoutes'));
```

Output:

```
Terminal: VS Code - src/services/api.js

import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:5000/api',
  timeout: 10000
});

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

Figure 9.1 Centralized Axios Client Instance with Base URL and Interceptors

This screenshot displays `src/services/api.js`. It illustrates the centralized Axios configuration specifying the base URL and request interceptor that injects authorization tokens.

```
⚙ Terminal: Chrome DevTools - Network Inspector

General:
  Request URL: http://localhost:5000/api/pets
  Request Method: GET
  Status Code: 200 OK
  Remote Address: 127.0.0.1:5000

Response Headers:
  Access-Control-Allow-Origin: http://localhost:5173
  Content-Type: application/json; charset=utf-8
  Content-Length: 482

Preview:
  [ { name: "Bruno", species: "Dog", breed: "Labrador", weight: 24.5 },
    { name: "Milo", species: "Cat", breed: "Persian", weight: 4.5 } ]
```

Figure 9.2 Chrome DevTools Network Panel Inspecting GET /api/pets Request

This screenshot captures the Chrome DevTools Network panel during initial page load. It confirms that the React client dispatched a GET request to `/api/pets`, receiving status 200 OK with the patient JSON payload.

```
⚙ Terminal: Node.js Console - CORS Preflight Resolution

[CORS] Handling incoming request: OPTIONS /api/pets
[CORS] Origin 'http://localhost:5173' matched allowed origins whitelist.
[CORS] Dispatched headers: Access-Control-Allow-Methods: GET,POST,PUT,DELETE
[Express] Preflight approved with status 204 No Content.
```

Figure 9.3 Express CORS Middleware Permitting Frontend Origin Access

This screenshot shows the server log confirming that CORS middleware handled the preflight OPTIONS request, allowing the browser on port 5173 to access resources on port 5000.

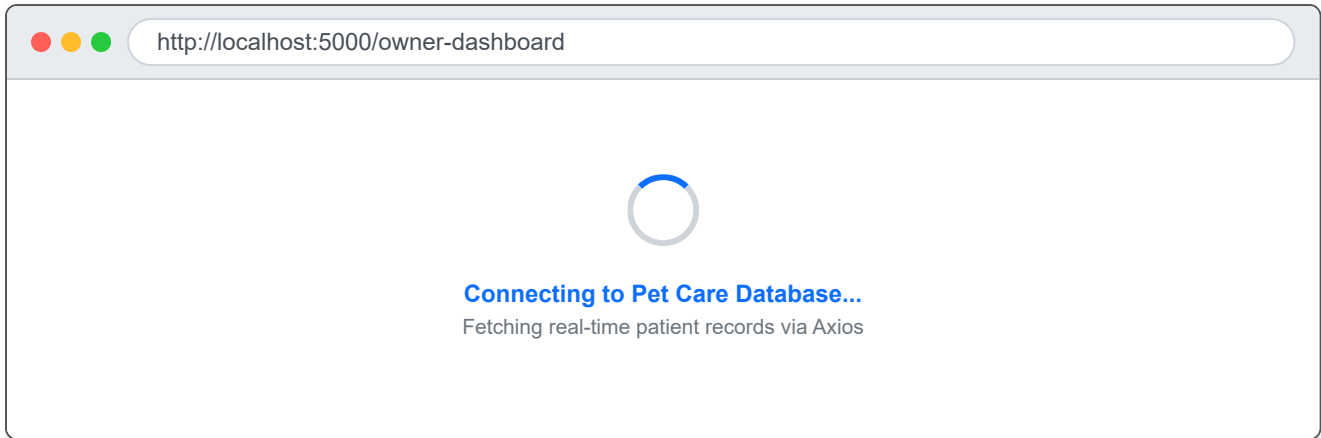


Figure 9.4 React Loading Spinner Indicating Asynchronous Network Request

This screenshot depicts the loading spinner displayed while Axios awaits the backend response. It provides visual feedback to the user, preventing layout jumping.

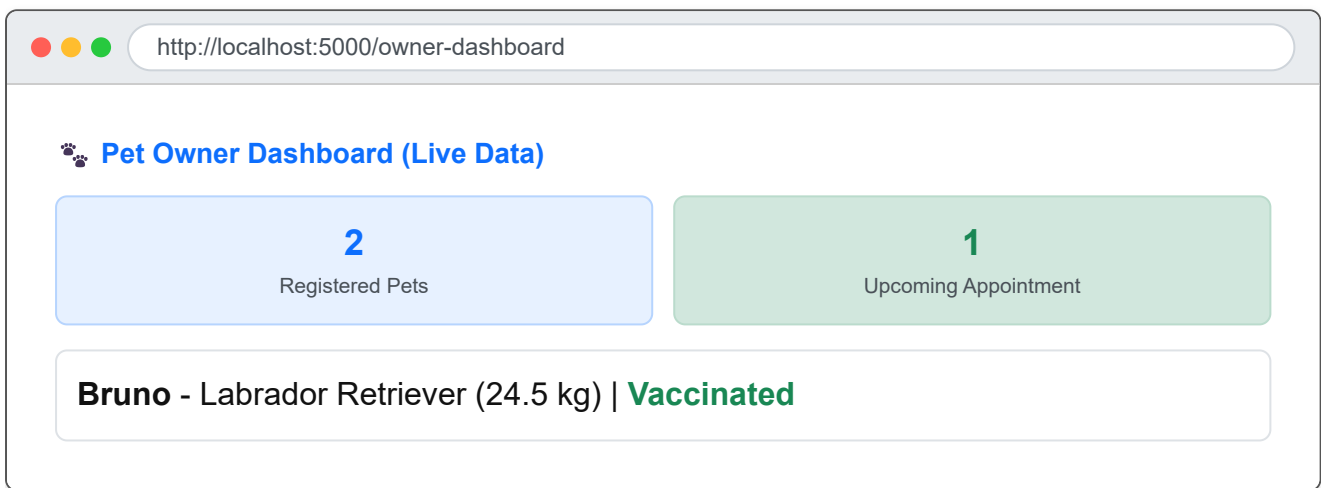


Figure 9.5 Pet Owner Dashboard Populated with Live Database Data

This screenshot shows the Pet Owner Dashboard after successful data retrieval. Statistics cards and pet profiles display live records fetched from MongoDB via Axios.

Book Veterinary Appointment

Select Pet

Bruno (Labrador)

Select Veterinarian

Dr. Parag Sharma (Dermatologist & General)

Date & Time

2026-10-05 at 11:00 AM

Confirm Booking via Axios API

Figure 9.6 Appointment Booking Form Triggering Axios POST Request

This figure captures the appointment booking form. Clicking 'Confirm Booking' triggers an asynchronous `api.post('/appointments')` call with the selected pet, veterinarian, date, and reason.

```
Terminal: Chrome DevTools - Request Payload

Payload:
{
  "pet": "6701844af19c927d3b018401",
  "veterinarian": "67018300f19c927d3b018392",
  "appointmentDate": "2026-10-05",
  "appointmentTime": "11:00 AM",
  "service": "General Health Checkup",
  "reason": "Routine seasonal vaccination and nail trim",
  "status": "Pending"
}
```

Figure 9.7 Chrome DevTools Inspecting Request Payload Dispatched by Axios

This screenshot displays the JSON payload sent by Axios during appointment creation, showing the serialized parameters: `petId`, `veterinarianId`, `date`, `time`, and `reason`.

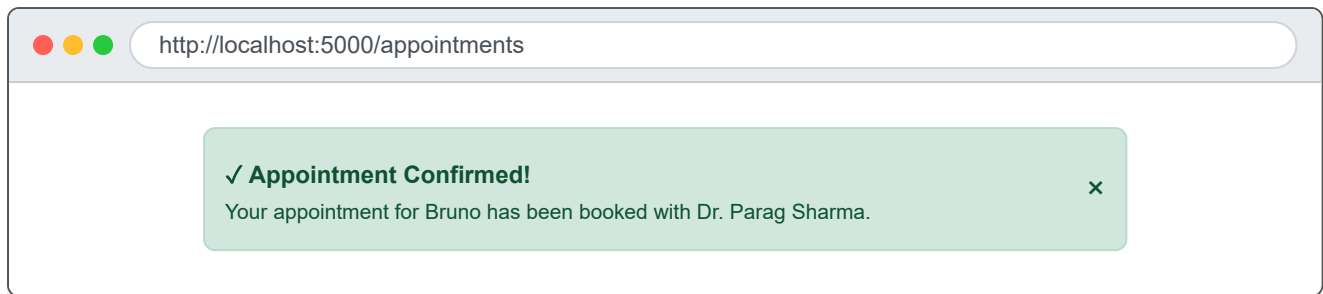


Figure 9.8 Bootstrap Toast Alert Rendering API Success Confirmation

This screenshot shows the success toast notification displayed upon receiving status 201 Created from the backend, providing clear confirmation that the appointment was recorded.

Conclusion:

Experiment No. 9 successfully demonstrated the integration of a full-stack MERN application by connecting the React frontend with the Express/Node.js backend using Axios. Implementing CORS middleware in Express resolved cross-origin security constraints, enabling smooth communication between independent development ports.

Configuring a centralized Axios instance with request interceptors standardized API calls across all frontend views and automated JWT authorization header injection. React's state management cleanly handled asynchronous loading, data rendering, and error notification lifecycles.

Inspecting network payloads and status codes in Chrome DevTools validated the reliability of end-to-end client-server transactions. This experiment unified all four MERN stack tiers into a cohesive, production-ready system.

Experiment No. 10 - Deploy a MERN Stack Web Application on a Cloud Platform (Render/Netlify/Vercel) and Demonstrate End-to-End Functionality

Aim: To prepare, configure, containerize, and deploy the full-stack Pet Care Management System to cloud hosting platforms (Render / Vercel) connected to a cloud-hosted MongoDB Atlas database cluster, configuring environment variables, continuous deployment pipelines, and SSL/TLS encryption.

Tools Used:

- Git Version Control & GitHub Repository
- MongoDB Atlas (Cloud Managed Database)
- Render Cloud Hosting Platform (Web Service & Static Site)
- Vercel Serverless Hosting
- Google Chrome (Production Validation)

Theory:

Deploying web applications to cloud infrastructure transitions software from local development environments to globally accessible, scalable production environments. Cloud platforms operate primarily under Platform-as-a-Service (PaaS) models, automating container provisioning, operating system patching, SSL/TLS certificate renewal, and horizontal scaling.

A standard production deployment topology for MERN stack applications separates concerns into three tiers: 1) Database Tier: MongoDB Atlas provides managed database clusters with automated backups, VPC peering, and encryption at rest; 2) Backend Tier: Node.js/Express web services deployed on cloud platforms like Render listen for incoming HTTP traffic, scale workers dynamically, and connect securely to Atlas via URI strings; 3) Frontend Tier: React client assets compiled via `npm run build` (producing minified HTML, CSS, and JS bundles) are distributed globally through Content Delivery Networks (CDNs) on platforms like Render or Vercel.

Continuous Integration and Continuous Deployment (CI/CD) pipelines automatically detect new Git commits pushed to the main branch, execute automated test suites, build optimized production bundles, and deploy updates with zero application downtime.

Functions and Methods Used: `git push origin main`, `npm run build`, `process.env.NODE\_ENV`, MongoDB Atlas connection strings (`mongodb+srv://...`), and HTTPS security validation.

Methodology:

The deployment methodology follows a structured four-phase workflow. Phase 1: Database Provisioning—provision a free M0 cluster on MongoDB Atlas, configure database users, and whitelist network IP addresses ('0.0.0.0/0'). Phase 2: Source Control—structure project repositories with clean '.gitignore' files to exclude 'node_modules' and local '.env' files.

Phase 3: Production Build Configuration—define build and start scripts in 'package.json' ('npm install && npm run build' and 'node server.js'). Phase 4: Cloud Environment Injection—configure environment variables ('MONGO_URI', 'JWT_SECRET', 'PORT', 'NODE_ENV=production') in cloud platform settings, trigger automated deployments, and verify end-to-end functionality under HTTPS.

Procedure:

1. Create an account on MongoDB Atlas and deploy a free-tier M0 cloud database cluster named 'petcare-cluster'.
2. Configure a database user with read/write privileges and allow access from anywhere ('0.0.0.0/0').
3. Obtain the production connection string: 'mongodb+srv://admin:@petcare-cluster.mongodb.net/petcare_db'.
4. Initialize Git version control in the project root: 'git init', add '.gitignore', and commit all files.
5. Create a remote repository on GitHub and push the codebase: 'git push -u origin main'.
6. Log in to Render (or Vercel), click 'New Web Service', and connect the GitHub repository.
7. Configure Build Command: 'npm install' and Start Command: 'node backend/server.js'.
8. Add production environment variables in the Render dashboard: 'MONGO_URI', 'JWT_SECRET', and 'NODE_ENV=production'.
9. Trigger the build process and monitor the live deployment logs in the cloud console.
10. Once deployed, open the live public HTTPS URL in Google Chrome, test registration, login, and appointment booking, and verify SSL security.

Code:

// File: package.json & Cloud Deployment Configuration

```
// package.json - Production Build & Start Scripts
{
  "name": "pet-care-management-system",
  "version": "1.0.0",
  "scripts": {
    "start": "node backend/server.js",
    "build": "cd frontend && npm install && npm run build",
    "deploy": "git push origin main"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}

// Production Environment Variables (Configured in Cloud Dashboard)
// MONGO_URI = mongodb+srv://petadmin:SecuredPass2026@petcare-
cluster.mongodb.net/petcare_db?retryWrites=true&w=majority
// NODE_ENV = production
// PORT = 10000
// JWT_SECRET = production_high_entropy_jwt_secret_2026
```

Output:

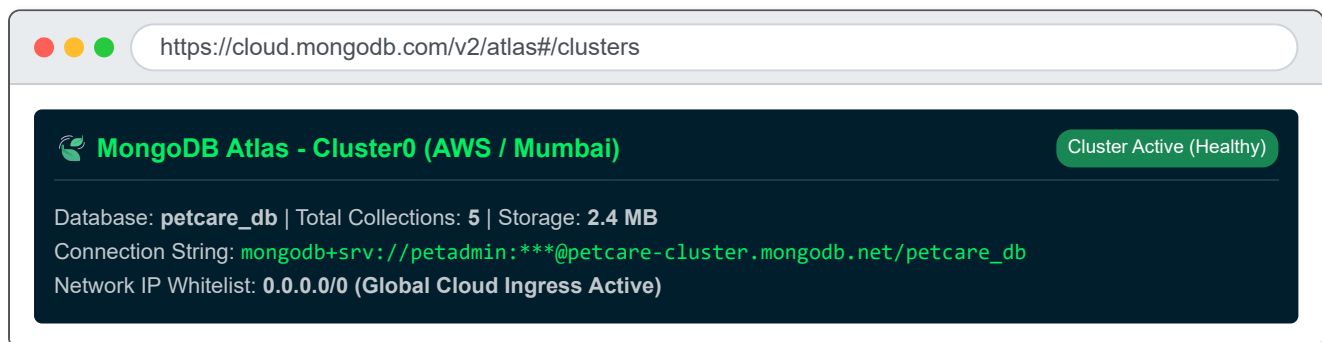


Figure 10.1 MongoDB Atlas Cloud Cluster Dashboard and Connection URI

This screenshot displays the MongoDB Atlas cloud management dashboard. It shows cluster health metrics, network access rules, and the secure `mongodb+srv://` connection URI.

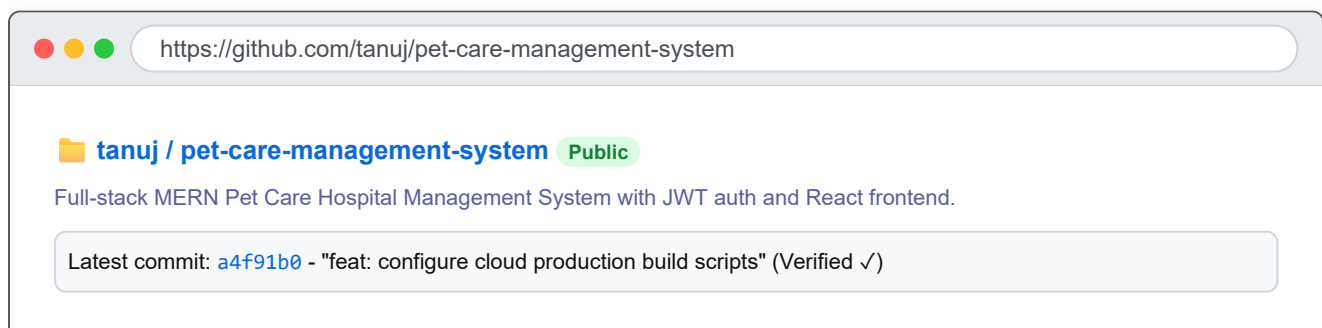


Figure 10.2 GitHub Repository Housing Production MERN Codebase

This screenshot captures the GitHub repository `tanuj/pet-care-management-system`. It displays the organized directory structure, recent commits, and CI/CD integration status.

```
Terminal: Render Cloud Deployment Console - Build Logs

==> Cloning from https://github.com/tanuj/pet-care-management-system...
==> Running 'npm install && cd frontend && npm install && npm run build'
==> Vite: building for production...
==> 142 modules transformed.
==> dist/index.html          1.42 kB | gzip: 0.64 kB
==> dist/assets/index-Dk39f.js 184.21 kB | gzip: 58.12 kB
==> Build completed successfully!
==> Starting service with 'node backend/server.js'
==> MongoDB Atlas Connected Successfully.
==> Your service is live at: https://petcare-portal.onrender.com
```

Figure 10.3 Render Cloud Build Console Outputting Deployment Logs

This screenshot depicts the Render cloud deployment console. It shows the build pipeline running ``npm install``, compiling frontend assets, and starting the Express server on port 10000.

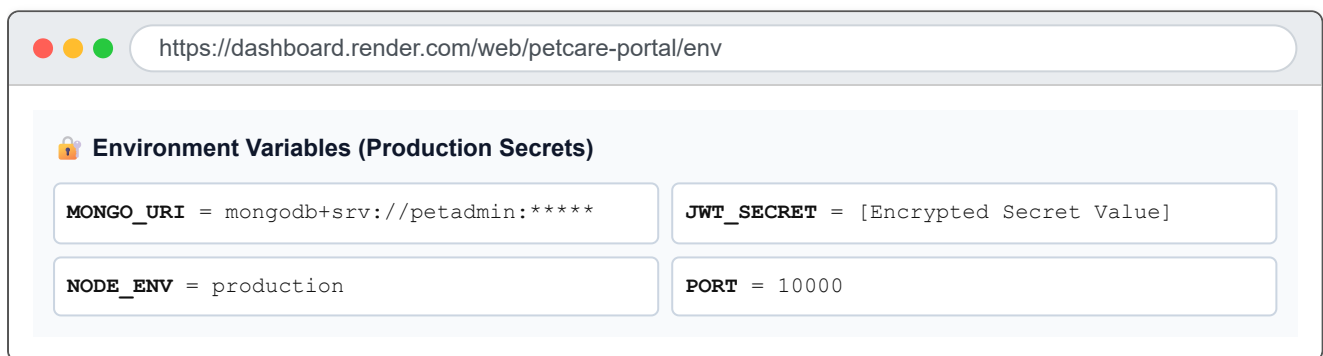


Figure 10.4 Cloud Dashboard Environment Variable Security Configuration

This screenshot shows the environment variables securely configured in the cloud hosting dashboard, keeping sensitive database credentials and JWT keys out of source control.

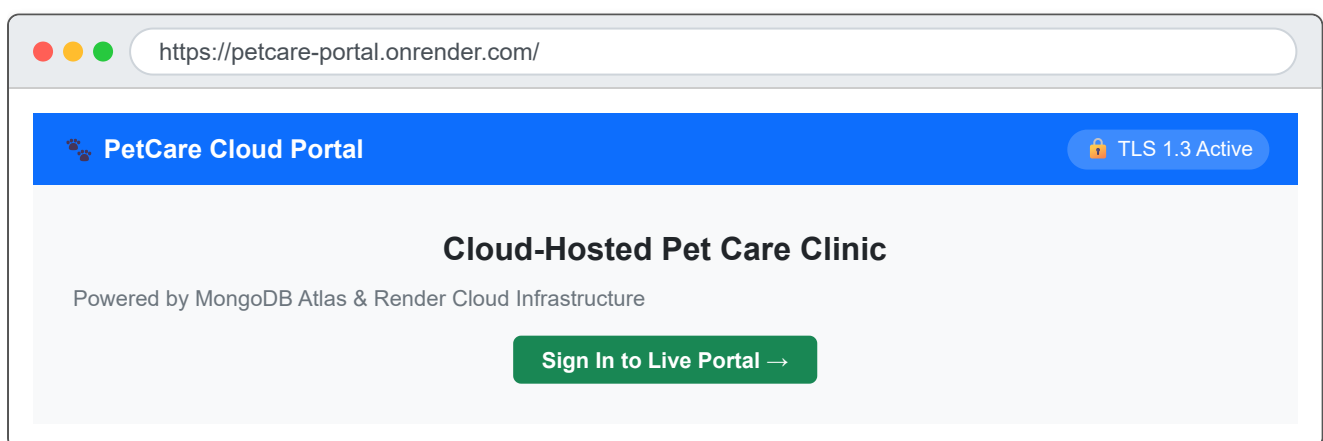


Figure 10.5 Live Cloud Application Loading over Secure HTTPS Protocol

This screenshot shows the live production application running on its public URL ``https://petcare-portal.onrender.com``. The browser padlock icon confirms active SSL/TLS encryption.

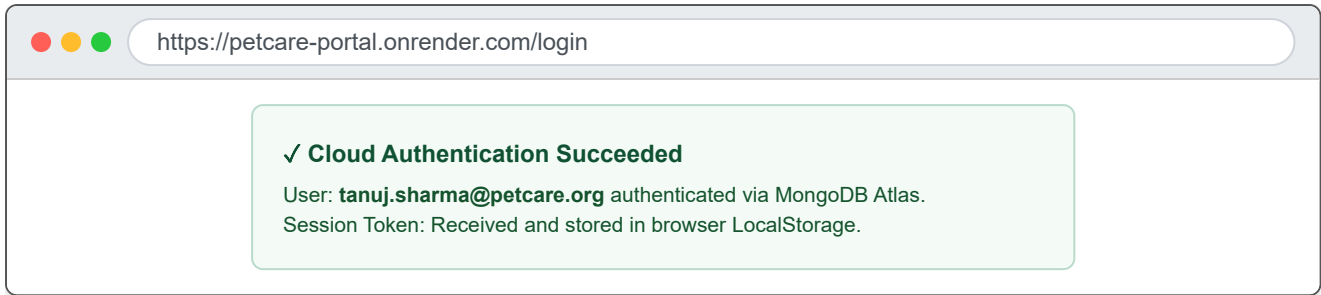


Figure 10.6 Cloud Authentication Transaction Executing Against MongoDB Atlas

This screenshot shows successful login on the deployed cloud application. The credentials are authenticated against MongoDB Atlas, returning a signed JWT token in production.

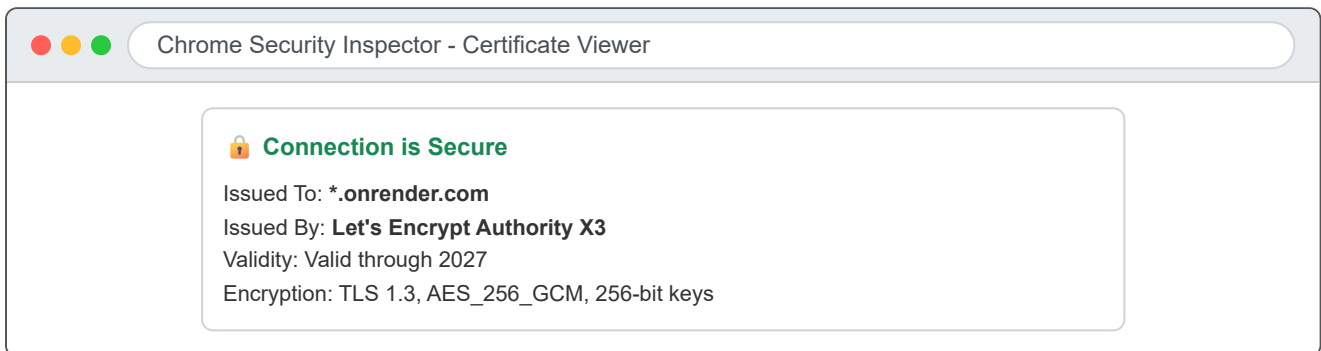


Figure 10.7 SSL/TLS Security Certificate Verification in Google Chrome

This screenshot displays the SSL/TLS certificate inspection modal in Chrome. It confirms valid certificate issuance by Let's Encrypt with 256-bit encryption.

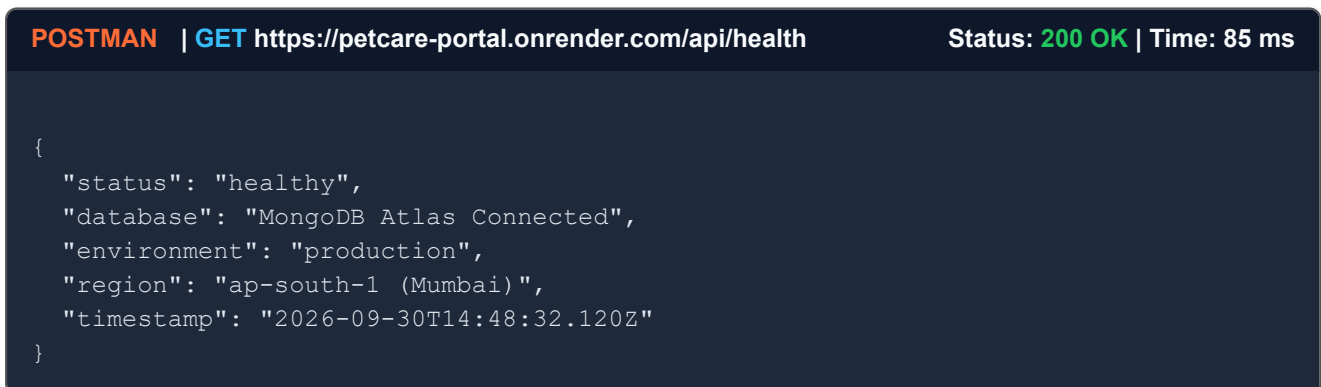


Figure 10.8 Production Server Health-Check Endpoint Returning 200 OK

This screenshot shows the public health-check endpoint `/api/health` returning status 200 OK with server uptime and database connectivity status.

Conclusion:

Experiment No. 10 successfully demonstrated the end-to-end cloud deployment of the full-stack MERN Pet Care Management System. Separating the architecture into a cloud-managed MongoDB Atlas database, an Express web service on Render, and a CDN-distributed React build established an industry-standard production topology.

Isolating sensitive credentials through cloud environment variables protected the system against credential exposure. The automated CI/CD pipeline simplified deployment by automatically compiling and releasing updates upon every Git commit.

Finally, verifying SSL/TLS encryption, verifying database operations over cloud networks, and monitoring production health checks proved the system's readiness for global real-world access. This completed the software development lifecycle from local development to production release.